



GitHub Copilot

GitHub Copilot 실무 활용

S.Pin Technology

lucas.lee@spintech.co.kr

```
log.html index.js
express = require('express');
router = express.Router();
User = require('../models/user');

// GET /register
router.get('/register', function(req, res, next) {
  return res.render('register', { title: 'Sign Up' });
});

// POST /register
router.post('/register', function(req, res, next) {
  if (req.body.email &&
      req.body.name &&
      req.body.favoriteBook &&
      req.body.password &&
      req.body.confirmPassword) {

    // confirm that user typed same password twice
    if (req.body.password !== req.body.confirmPassword) {
      var err = new Error('Passwords do not match.');
```

AGENDA

오늘 일정

세션	내용
Introduction & Environment Check	오늘 배울 것을 먼저 짚고, 각자 PC에서 실습 환경과 저장소가 준비됐는지 확인합니다.
Copilot in VS Code 기능 Overview & Agent Role Strategy	Code Completion과 Next edit suggestions를 화면으로 보고, Role과 Harness, 모델을 무엇을 기준으로 고르는지 정리합니다.
Prompt Engineering 기초 & 기본 명령어	같은 일을 시키는 두 요청을 나란히 놓고 비교한 뒤, 작성 원칙 여덟 가지와 입력창의 @ # / 를 익힙니다.
Context 설계: 지시문 파일과 Agent Skills	모델이 답할 때 무엇을 함께 읽는지 일곱 가지로 나눠 보고, 그중 우리가 정할 수 있는 것을 파일로 적어 둡니다.
Hands-On Lab 1: Requirement → Plan → Code	/init 으로 지시문을 만들고, /plan 으로 받은 계획을 읽고 되물은 뒤 실행을 맡깁니다.
Hands-On Lab 2: Refactoring, Testing & Debugging	고치기 전에 지금 동작을 테스트로 기록해 두고 , 그 위에서 구조를 바꿉니다. 동작이 달라지는 변경은 사람이 판단합니다.
Hands-On Lab 3: Context & Token 효율화	한 세션에서 Credit이 얼마나 쓰이는지 직접 읽고, Tool 구성만 바꿔 두 번 실행해 차이를 비교합니다.
MCP 개념 & 사내 시스템 연계 방안	외부 시스템을 Tool로 잇는 구조를 보고, 서버 등록 방법과 사내 Jira, Collab 연동을 다룹니다.
Capstone Hands-On Lab	남겨 둔 버그 하나를 실패하는 테스트부터 쓰고 계획을 받아 고치는 데까지, 전체 과정을 해결해 봅니다.
Wrap-up & Q&A	오늘 쓴 기능을 한 장으로 정리하고, 요청마다 정해야 하는 다섯 가지를 짚습니다.

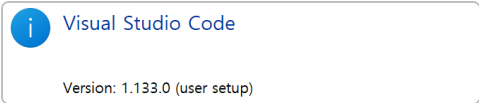
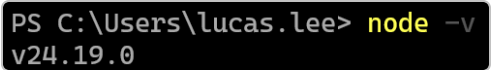
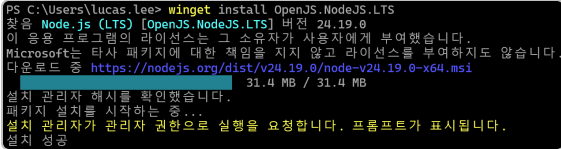


Introduction & Environment Check

오늘의 목표

항목	내용
Agent Role	일의 성격에 따라 Agent와 Plan을 골라 씁니다. 바로 맡길 때와 계획부터 받을 때를 구분합니다.
Prompt	목표, 범위, 완료 조건을 갖춰 요청합니다. @ # / 로 필요한 것만 정확히 넘깁니다.
Context 설계	팀 규약을 지시문 파일에 남깁니다. 매번 설명하지 않아도 Copilot이 따릅니다.
계획과 실행	요구사항을 바로 코드로 바꾸지 않습니다. 계획을 사람이 검토한 뒤 실행을 맡깁니다.
회귀 테스트	현재 동작을 고정한 뒤 구조를 바꿉니다. 구조만 바꿨다는 것을 증명할 수단입니다.
비용과 확장	Context와 토큰 사용량을 확인해 줄입니다. MCP로 사내 시스템까지 연결합니다.

실습 환경 점검

확인 항목	확인 방법	조치 방법
VS Code 버전	Help > About에서 Version 확인 터미널이면 <code>code --version</code> 1.130 이상이면 통과 	1.130 미만이면 창을 전부 닫아 VS Code를 완전히 종료한 뒤 다시 실행합니다. 그래도 낮으면 code.visualstudio.com 에서 받아 덮어쓰기로 설치합니다.
Copilot	Ctrl+Alt+I 로 Chat을 열고 아무 문장이나 보냅니다 답변이 오면 통과	답이 없으면 상태 표시줄 Copilot 아이콘을 누릅니다. Use AI Features가 보이면 로그인 전입니다. 눌러서 회사 GitHub 계정으로 로그인합니다.
Node.js	터미널에서 <code>node --version</code> 결과가 v22 이상 	낮거나 없으면 <code>winget install OpenJS.NodeJS.LTS</code> 또는 nodejs.org에서 설치한 뒤 터미널을 새로 엽니다 
실습 저장소	ZIP을 푼 폴더를 열고 <code>npm test</code> 통과 <code>npm install</code> 은 실행하지 않습니다.	<code>vitest</code> 를 찾을 수 없다고 나오면 <code>node node_modules/vitest/vitest.mjs run</code> 으로 같은 테스트를 돌립니다.

실습 저장소

차량 견적을 계산하는 작은 JavaScript 프로젝트입니다. 외부 프레임워크를 쓰지 않고, 테스트는 Vitest 로 돌립니다.

소스코드 위치: <https://github.com/yoonyeong0219/copilot-workshop-js>

경로	내용
<code>src/quote.js</code>	견적 계산. 함수 하나가 110줄입니다. Hands-On Lab 2에서 고칠 대상입니다.
<code>src/catalog.js</code>	트림, 옵션, 패키지 기준 데이터입니다.
<code>src/format.js</code>	견적 결과를 문자열로 바꿉니다. 테스트가 이미 있어 작성 스타일의 기준이 됩니다.
<code>src/warmup.js</code>	Code Completion 실습 전용입니다. 테스트가 없어 마음껏 고쳐도 됩니다.
<code>src/demo.js</code>	npm start 로 견적을 계산해 화면에 출력합니다. Hands-On Lab 1에서 함께 고칩니다.
<code>test/format.test.js</code>	저장소에 있는 유일한 테스트입니다. 작성 스타일의 기준이 됩니다.
<code>docs/hol1-requirement.md</code>	Hands-On Lab 1 요구사항 문서. 끝에 확인이 필요한 항목 세 가지를 남겨 두었습니다.

※ 이 코드에는 고칠 곳이 일부러 남아 있습니다. 잘 짜인 예제가 아니라, 실무에서 만나는 코드에 가깝게 만들어 두었습니다.

실습 저장소 – 기준 데이터

계산에 쓰는 값은 catalog.js 한 파일에 모여 있습니다. 순수 데이터만 담고 계산은 하지 않습니다.

src/catalog.js – 트림은 차량 등급입니다

```
export const TRIMS = {
  standard: { name: '스탠다드', basePrice: 28_900_000 },
  premium: { name: '프리미엄', basePrice: 34_500_000 },
  signature: { name: '시그니처', basePrice: 41_200_000 },
};
```

개별 옵션과 패키지 – 패키지는 포함 옵션을 묶어 할인합니다

```
export const OPTIONS = {
  hud: { name: '헤드업 디스플레이', price: 980_000 },
  'premium-audio': { name: '프리미엄 오디오', price: 1_180_000 },
  sunroof: { name: '파노라마 선루프', price: 1_350_000 }, // 그 밖에 세 가지가 더 있습니다
};

export const PACKAGES = {
  tech: { name: '테크', includes: ['hud', 'premium-audio'], discountRate: 0.15 },
};
```

※ 지역별 탁송료 표도 같은 파일에 있습니다. 기준 데이터를 한곳에 모아 둔 것이 이 저장소의 전제이며, 이 전제가 Hands-On Lab 2에서 다시 나옵니다.

실습 저장소 – 견적이 나오는 순서

calculateQuote 하나가 여섯 단계를 차례로 밟습니다. 앞 단계가 더한 금액을 뒤 단계가 그대로 받습니다.

순서	하는 일
1	주문에 적힌 트림을 TRIMS 에서 찾아 그 차값을 합계의 출발점으로 잡습니다.
2	따로 고른 옵션이 있으면 OPTIONS 에서 가격을 찾아 하나씩 더합니다.
3	패키지는 포함된 옵션 가격을 먼저 합한 뒤 할인율만큼 깎아서 더합니다.
4	여기까지 나온 금액에 세금을 붙입니다. 개별소비세 5%, 교육세는 개별소비세의 30%, 부가가치세 10%입니다.
5	차를 받을 지역에 따라 택송료를 더합니다.
6	법인 고객이면 할인하고, 타던 차를 넘겼다면 보상 판매액을 뺍니다.

실습 저장소 – 주문과 견적서

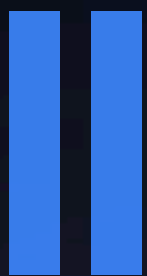
주문은 객체 하나입니다. src/demo.js 에 적어 두었고, npm start 로 결과를 볼 수 있습니다.

src/demo.js – 계산할 주문입니다

```
const order = {
  trim: 'premium',           // 트림
  options: ['sunroof'],      // 따로 고른 옵션
  packages: ['tech'],        // 고른 패키지
  region: 'jeju',            // 차를 받을 지역
  customerType: 'individual', // 개인인지 법인인지
  tradeInValue: 3_000_000,    // 타던 차를 넘긴 값
};
```

npm start – 위 주문을 계산한 결과

프리미엄	34,500,000원	부가가치세	4,013,559원
파노라마 선루프	1,350,000원	탁송료	520,000원
테크 패키지	1,836,000원	보상 판매	-3,000,000원
개별소비세	1,884,300원		
교육세	565,290원	합계	41,669,149원



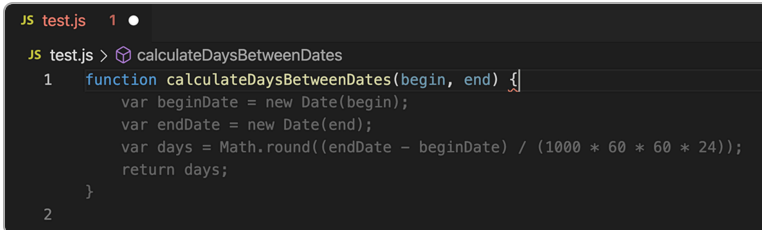
Copilot in VS Code 기능 Overview & Agent Role Strategy

Code Completion과 Next edit suggestions

Code Completion

커서 다음에 올 코드

- 생김새** 흐린 회색 글자로 표시됩니다. 한 줄만 제안될 때도 있고, 함수 전체가 제안될 때도 있습니다.
- 받기** **Tab** 을 누르면 제안된 코드가 삽입되며, 계속 입력하면 제안이 사라집니다.
- 일부만** 제안 전체를 받지 않으려면 **Ctrl+→** 를 누릅니다. 한 단어씩 또는 한 줄씩 삽입됩니다.
- 잠시 멈춤** 제안이 방해되면 상태 표시줄 **Copilot** 메뉴에서 **Snooze** 를 누릅니다. 한 번에 5분씩 멈추고, **Cancel Snooze** 로 다시 켵니다.



```
JS test.js 1 •
JS test.js > calculateDaysBetweenDates
1 function calculateDaysBetweenDates(begin, end) {
  var beginDate = new Date(begin);
  var endDate = new Date(end);
  var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));
  return days;
}
2
```

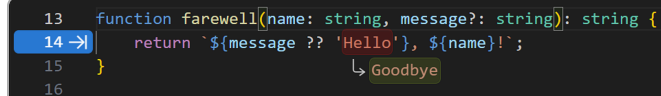
회색 글자로 함수 본문이 통째로 나온 모습

※ 둘 다 **Chat** 창을 열지 않고 편집기에서 바로 동작합니다. AI Credit을 소비하지 않으며, 유료 플랜(Copilot Business, Copilot Enterprise 등)에서 무제한으로 사용 가능합니다.

Next edit suggestions

다음에 고칠 곳

- 생김새** 편집기 왼쪽 줄 번호 옆에 화살표가 표시됩니다. 커서 기준으로 어느 방향에 고칠 곳이 있는지 가리킵니다.
- 받기** **Tab** 을 누르면 커서가 그 자리로 이동하고, **Tab** 을 한 번 더 누르면 제안된 코드가 삽입됩니다.
- 쓰는 때** 오타를 고치거나, 하려던 방식을 바꾸거나, 구조를 바꿀 때 사용합니다. 이름을 한 번 바꾸면 그 이름이 쓰인 자리마다 따라옵니다.
- 설정** 제안이 뜨지 않으면 상태 표시줄의 **Copilot** 메뉴를 열어 **Next Edit Suggestions**가 **켜져 있는지** 확인합니다.



```
13 function farewell(name: string, message?: string): string {
14 →   return `${message ?? 'Hello'}, ${name}!`;
15   }
16   ↪ Goodbye
```

화살표가 다음에 고칠 줄을 가리킵니다

warmup.js 실습

src/warmup.js 를 여십시오. 파일 안에 실습 1, 2, 3 이 주석으로 표시되어 있습니다.

실습 1 – Code Completion

addVat 의 return 한 줄을 지우고 빈 줄에서 기다립니다. 흐린 회색 글자가 뜨면 Tab 으로 받습니다.

```
/**
 * 실습 1 – 코드 완성
 *
 * 아래 함수 본문(return 한 줄)을 통째로 지우고, 빈 줄에서 잠깐 기다려 보십시오.
 * 흐린 회색 글자로 제안이 붙습니다. Tab 을 누르면 들어갑니다.
 * 한 단어씩 받고 싶으면 Ctrl + → 를 쓰십시오.
 */
export function addVat(supply) {
  return Math.round(supply + supply * VAT_RATE);
}
```

실습 2 – Next edit suggestions

매개변수 supply 를 amount 로 바꿉니다. 본문에서 다섯 번 쓰이므로 화살표를 Tab 으로 따라잡니다.

```
/**
 * 실습 2 – NES
 *
 * 아래 함수의 매개변수 이름 supply 를 amount 로 바꿔 보십시오.
 * 본문에서 supply 를 다섯 번 쓰고 있으므로, 첫 자리를 고치면
 * 편집기 왼쪽 줄 번호 옆에 화살표가 뜨고 나머지 자리를 따라옵니다.
 * Tab 으로 그 자리로 가고, 한 번 더 Tab 으로 받습니다.
 */
export function describeSupply(supply) {
  if (typeof supply !== 'number' || Number.isNaN(supply)) {
    throw new TypeError('공급가는 숫자여야 합니다');
  }
  if (supply <= 0) {
    return '공급가가 0원 이하입니다';
  }
  const vat = Math.round(supply * VAT_RATE);
  const total = supply + vat;
  return `공급가 ${supply}원, 부가세 ${vat}원, 합계 ${total}원`;
}
```

실습 3 – 주석으로 함수 만들기

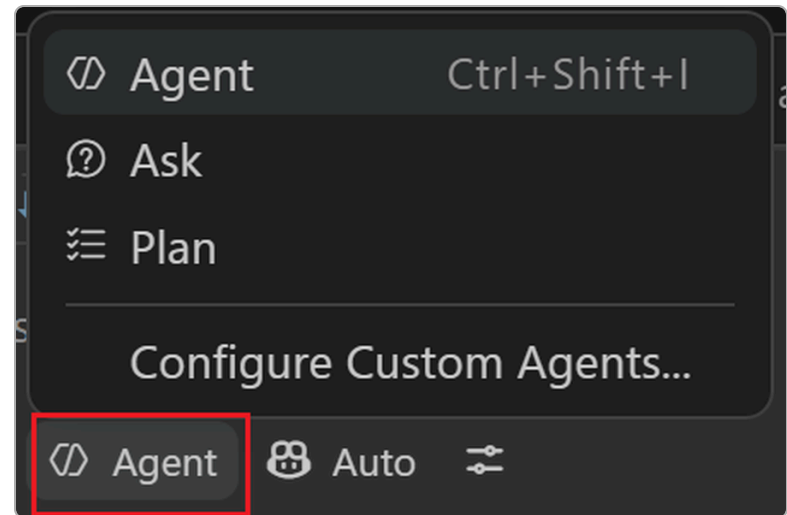
주석 다음 줄에서 Enter 를 누르고 기다립니다. 함수 하나가 통째로 제안됩니다.

```
/**
 * 실습 3 – 주석으로 함수 만들기
 *
 * 아래 주석 다음 줄에서 Enter 를 누르고 잠깐 기다리십시오.
 * 함수 하나가 통째로 제안됩니다.
 */
// 공급가 배열을 받아 부가세를 포함한 합계 금액을 돌려줍니다
```

※ 세 실습 모두 Chat 창을 열지 않고 편집기에서 진행합니다. warmup.js 에는 테스트가 걸려 있지 않아 무엇을 고쳐도 npm test 는 그대로 통과합니다.

내장 Agent Role 세 가지

Chat 창의 agent picker에서 사용할 agent를 선택할 수 있습니다.



Chat 입력창 왼쪽 아래 picker

01

Ask – 읽고 이해하는 역할

동작 코드를 읽고 설명합니다. 파일 수정은 Agent가 맡습니다.

상황 코드 기반 질의가 필요하거나 기존 코드 분석이 필요할 때 사용합니다.

02

Plan – 계획을 세우는 역할

동작 구현을 시작하기 전에 구조화된 구현 계획을 생성합니다. Start Implementation을 누르기 전까지 코드는 그대로입니다.

상황 요구사항 기반 구현 계획을 세울 때 사용합니다.

이유 엉뚱한 문제를 푸는 상황을 미리 막고, 버릴 코드에 Credit을 쓰지 않게 됩니다.

03

Agent – 직접 고치는 역할

동작 스스로 계획을 세워 파일을 편집하고, 명령을 실행하고, 결과를 보고 다시 시도합니다.

상황 수정 방향성이나 방법이 이미 정해져 있을 때 사용합니다.

Role 선택 기준

상황	선택
이 코드가 무슨 일을 하는지부터 알고 싶습니다	Ask로 물어봅니다. 고치기 전에 읽는 단계입니다.
손델 범위가 넓거나 어디부터 볼지 모르겠습니다	Plan으로 계획을 받고 검토한 뒤 수정을 진행합니다.
고치되 코드가 바로 바뀌는 것은 원하지 않습니다	Plan을 씁니다. 계획을 만드는 동안 코드는 건드리지 않습니다.
고칠 파일과 방법이 이미 정해져 있습니다	Agent에게 바로 맡깁니다.
한 줄 고치는 정도입니다	Chat을 열 것 없이 Code Completion과 Next edit suggestions로 충분합니다.

Agent가 일하는 순서

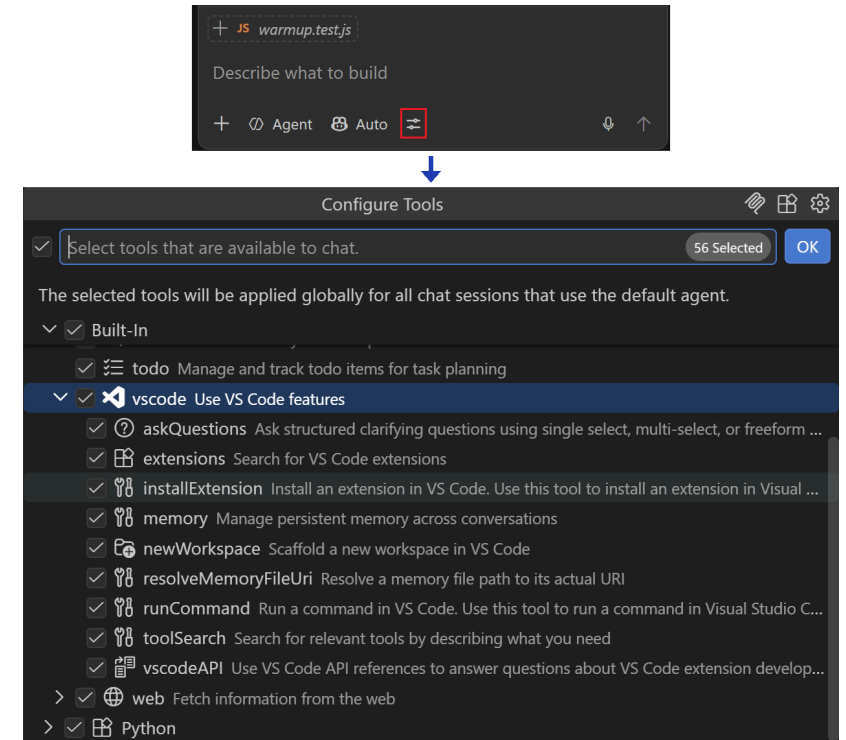
Agent는 한 번 답하고 끝나지 않습니다. 아래 단계를 돌면서 결과를 보고 다시 시도합니다.

단계	동작	실제 화면
01	받은 목표를 실행 가능한 단계로 분해합니다.	일이 복잡하면 진행 상황을 todo list 로 보여줍니다.
02	코드를 읽고 검색해 무엇을 고쳐야 할지 파악합니다.	Agent가 무엇을 했는지 한 줄씩 남습니다.
03	승인이 필요한 Tool이면 멈추고 사람에게 묻습니다.	확인 창이 뜨고, 답할 때까지 기다립니다.
04	파일을 편집하고, 필요하면 터미널 명령을 실행합니다.	편집한 파일이 변경 목록 에 모입니다.
05	테스트를 돌리고 그 출력을 읽습니다.	터미널 출력이 그대로 표시됩니다.
06	결과가 어긋나면 다시 읽고 고칩니다. 사람이 시키지 않아도 되풀이합니다.	통과할 때까지 이어집니다.

Agent가 부르는 Tool

Tool은 Agent가 직접 부르는 기능입니다. 파일을 읽고, 코드를 검색하고, 터미널 명령을 실행하는 일이 모두 Tool 호출입니다.

종류	내용	다루는 곳
내장 Tool	VS Code에 이미 들어 있습니다. 파일 읽기, 코드 검색, 터미널 실행, 파일 편집을 설치 없이 씁니다.	오늘 실습 전체
MCP Tool	MCP 서버를 등록하면 그 서버가 제공하는 기능이 목록에 더해집니다.	MCP 챕터
확장 Tool	확장을 설치하면 그 확장이 제공하는 기능이 목록에 더해집니다.	오늘 쓰지 않습니다



아이콘을 누르면 목록이 열립니다

※ Agent는 커 둔 Tool 중에서만 고릅니다. 지금 작업에 관계없는 Tool은 꺼 두십시오.

Agent loop을 돌리는 Harness

Harness는 Agent가 일하는 여섯 단계를 실제로 돌리는 런타임입니다. 모델은 추론만 하고 나머지는 harness가 맡습니다.

HARNESS

내용

Local

VS Code가 직접 돌립니다. 열어 둔 파일과 터미널 출력을 그대로 보면서 일하고, 변경은 항상 현재 Workspace에 적용됩니다. 오늘 쓰는 harness입니다.

Copilot

이름과 달리 내 PC에서 돌립니다. VS Code가 아니라 Agent Host라는 별도 프로그램이 세션을 맡아서, VS Code를 달아도 작업이 이어집니다.

Cloud

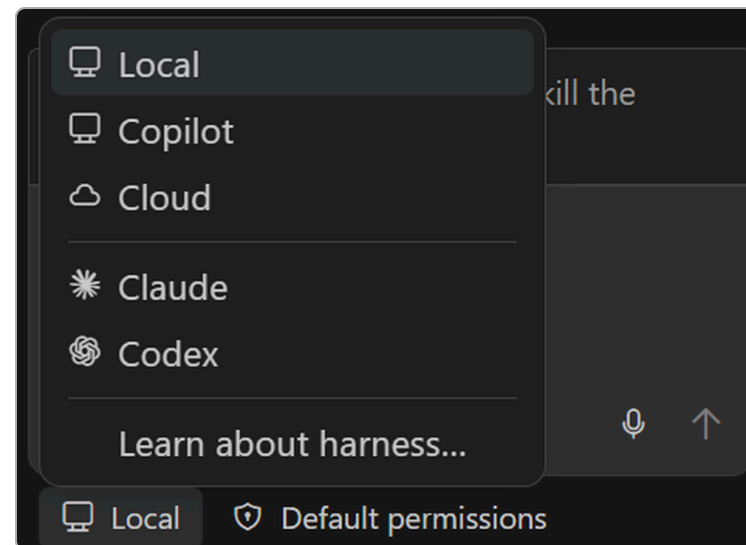
원격 인프라에서 실행되고 GitHub 저장소가 필요합니다. branch 작업 후 Pull Request를 생성하며, 로컬 편집기의 Context와 터미널 출력, 확장이 제공하는 Tool은 쓰지 못합니다.

Claude

Anthropic의 Claude Agent SDK로 세션을 돌립니다. 기본으로 켜져 있고 Copilot 구독만 있어도 씁니다. Anthropic API 키로 직접 붙을 수도 있습니다.

Codex

OpenAI의 코딩 Agent인 Codex를 씁니다. 기본 목록에 없어서 전용 확장을 설치해야 나타납니다.



목록 구성은 버전과 계정에 따라 달라집니다

※ 오늘은 Local 하나만 씁니다. 모델을 Claude로 고르는 것과 Claude harness를 고르는 것은 다릅니다. 앞은 추론만, 뒤는 런타임까지 바꿉니다.

작업에 맞는 모델 선택

모델마다 한 번에 볼 수 있는 정보의 양(Context Window)과 추론 방식이 다릅니다. 작업 유형에 맞는 모델을 골라야 비용과 업무 효율을 함께 높일 수 있습니다.

작업 유형

해당 모델

강점

빠르고 반복적인 작업

Claude Haiku 4.5

가벼운 코딩 질문에 빠르고 안정적으로 답합니다.

범용 코딩

GPT-5 mini, Claude Sonnet 4.5 / 4.6

빠르고 정확한 코드 완성과 설명. Sonnet은 Agent 작업까지 포함합니다.

깊은 추론과 Debugging

GPT-5.4 / 5.5, Claude Opus 4.7, Gemini 3.1 Pro

여러 단계에 걸친 문제 해결과 아키텍처 수준의 코드 분석.

Agent 개발

GPT-5.3-Codex, GPT-5.4 mini

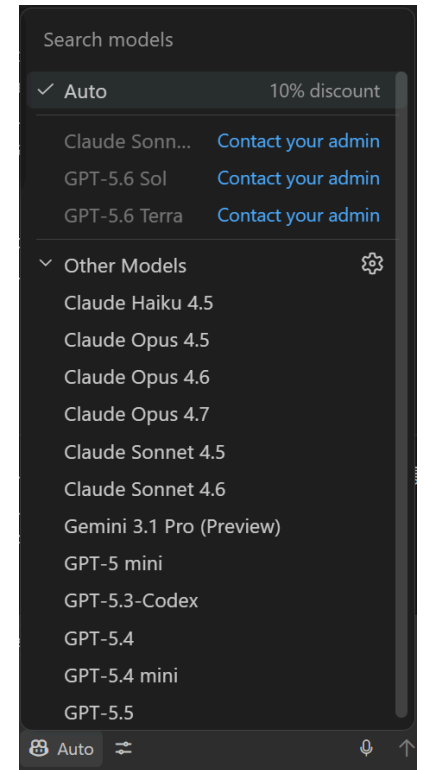
코드베이스 탐색. 특히 **grep** 계열 도구를 쓸 때 효과가 큼니다.

※ 목록은 조직이 허용한 범위와 시점에 따라 달라집니다. 모델은 세션을 시작할 때 정하고 중간에 바꾸지 마십시오. 앞선 요청과 겹치는 부분을 재사용하던 이점(cache)이 모델을 바꾸면 사라집니다.

Chat 입력창의 Model Picker

Chat 입력창 아래에서 열립니다. 목록에 나오는 표시는 세 가지입니다.

화면 표시	뜻
Auto	작업 복잡도와 모델 가용성을 보고 요청마다 알맞은 모델을 자동으로 고릅니다. 유료 플랜(Copilot Business, Copilot Enterprise 등)에서는 모델 비용 할인이 적용됩니다.
Contact your admin	조직 정책으로 잠긴 모델입니다. 목록에는 보이지만 고를 수 없습니다.
Other Models	지금 계정으로 고를 수 있는 모델 전부입니다. 작업 유형에 맞춰 고릅니다.



Chat 입력창 아래 Model Picker

Agent 실습

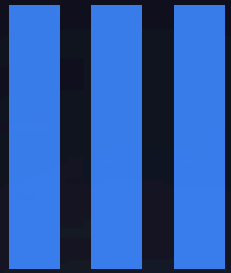
앞에서 손본 `src/warmup.js` 를 그대로 씁니다. Chat 을 열고 agent picker에서 **Agent** 를 고릅니다.

단계	내용
01	Ctrl+Alt+I 로 Chat 을 열고, agent picker가 Agent 인지 확인한 뒤 아래 한 줄을 보냅니다.
02	Agent 가 파일을 고치기 전에 승인을 물으면 그대로 진행합니다.
03	Agent가 테스트를 직접 실행합니다. 대화에 남은 출력에서 통과 여부를 확인하고, 바뀐 파일을 diff 로 훑습니다.
04	확인이 끝나면 탐색기에서 <code>test/warmup.test.js</code> 를 지웁니다.

01단계에 넣을 요청

`warmup.js` 의 `addVat` 에 대한 테스트를 `test/warmup.test.js` 에 추가해 주세요.

※ `npm test` 를 직접 칠 필요가 없습니다. **Agent가 스스로 돌리고 결과를 보고합니다.** 다만 새로 생긴 `test/warmup.test.js` 는 지워야 Hands-On Lab 10이 모두 같은 상태에서 시작합니다.



Prompt Engineering 기초 & 기본 명령어

Prompt의 두 가지 형태

Prompt는 사용자가 Copilot에 보내는 요청입니다.
Chat에 입력한 질문이거나, 완성해 달라고 남겨 둔 코드 조각입니다.

CHAT REQUEST

입력창에 쓰는 문장

형태	여러 문장을 이어 붙일 수 있고, 대상 파일을 참조로 함께 붙일 수도 있습니다.
예	src/quote.js 의 calculateQuote 에서 세금 계산 부분을 분리해 주세요
쓰는 때	대상과 조건을 명시적으로 적어야 하는 작업입니다. 수정, 계획 수립, 테스트 생성이 여기에 해당합니다.

CODE COMPLETION

코드에 남겨 두는 요청

형태	Chat을 열지 않습니다. 커서 바로 위의 주석과 함수 시그니처가 요청 역할을 합니다.
예	// 공급가 배열을 받아 합계 금액을 돌려줍니다 한 줄을 쓰고 기다립니다.
쓰는 때	지금 쓰고 있는 코드의 다음 몇 줄을 이어 받을 때 사용합니다. Chat 을 열지 않고 편집기에서 그대로 진행합니다.

※ 어느 쪽이든 무엇을 적고 무엇을 뺄지는 전부 사용자가 정합니다. 이 챕터는 그 문장을 어떻게 구성할지 다룹니다.

요청 문장이 정하는 범위

Agent가 어디까지 고칠지와 언제 멈출지는 요청 문장에 적힌 대로 정해집니다.
적지 않은 부분은 Agent가 지금 편집 중인 파일과 직전 대화를 보고 스스로 채웁니다.

BEFORE

이 코드 Refactoring 해 주세요

대상도 완료 조건도 없습니다. 지금 편집 중인 파일과 직전 대화만 보고 고칠 곳을 정하므로, 의도하지 않은 파일까지 바뀌고 어디까지 검토해야 하는지도 알 수 없습니다.

AFTER

src/quote.js 의 calculateQuote 에서

대상

세금 계산 부분을 별도 함수로 분리해 주세요.

할 일

함수 시그니처는 그대로 두고,

건드리지 말 것

npm test 가 전부 통과해야 합니다

완료 조건

※ 요청 한 문장이 diff 크기를 정합니다. 대상이 좁아야 검토가 끝나고, 완료 조건이 있어야 Agent가 그 명령을 직접 실행해 통과할 때까지 고칩니다.

Prompt 작성 원칙 여덟 가지

GitHub이 제시하는 Prompt 작성 원칙입니다.

이전 페이지의 AFTER 요청은 이 중 넷을 지킨 문장이었습니다.

원칙	내용
넓게 시작해 좁힌다	무엇을 만들려는지 한 문장으로 먼저 적고, 입력과 출력, 예외 처리 같은 조건을 그 아래에 줄로 나열합니다.
예시를 준다	입력 데이터, 기대 출력, 구현 예시를 함께 제시합니다. 단위 테스트도 예시가 됩니다.
큰 일은 쪼갬다	한 번에 파일 하나, 함수 하나 수준으로 끊습니다. 변경이 커질수록 사람이 diff를 끝까지 읽지 못합니다.
모호한 말을 피한다	"이게 뭐야" 대신 대상을 이름으로 부릅니다. 낯선 라이브러리는 무엇을 하는지 덧붙입니다.
관련 코드를 지정한다	관련 파일은 열고 관계없는 파일은 닫습니다. # 로 직접 붙이면 더 확실합니다.
고쳐 가며 반복한다	원하는 답이 아니면 새 대화를 열지 말고 같은 세션에서 이어서 고쳐 말합니다. 앞의 Context가 남아 있어 짧게 말해도 통합니다.
이력을 정리한다	새 작업은 새 스레드에서 시작하고, 쓸모없어진 요청은 지웁니다.
코드 자체를 좋게 둔다	Copilot은 주변 코드의 패턴을 따라갑니다. 네이밍과 구조가 일관된 파일에서는 생성 결과도 그 규칙을 지킵니다.

입력창의 @ # / 세 기호

@ 는 답할 상대를, # 는 Context에 넣을 대상을, / 는 자주 하는 작업을 가리킵니다.

기호	역할	용도
@	특정 주제를 담당하는 참여자 에게 요청을 넘깁니다.	VS Code 설정, 터미널 명령, 저장소 활동처럼 코드가 아닌 것 을 물을 때 사용합니다.
#	파일, 폴더, 심볼, 웹 문서를 Context 에 추가합니다.	대상을 좁힐 때 사용합니다. 오늘 실습에서 가장 자주 씁니다.
/	자주 하는 작업의 단축 표기 입니다.	테스트 생성, 오류 수정처럼 같은 문장을 반복해 쓸 때 사용합니다.

VS Code에 내장된 **대표적인 @ 참여자** 셋입니다. 확장을 설치하면 참여자가 늘어납니다.

참여자	답하는 범위	입력 예시
@github	저장소, 이슈, Pull Request 등	@github 나에게 할당된 열린 PR을 모두 알려 주세요
@terminal	통합 터미널과 shell 명령	@terminal 이 Workspace에서 가장 큰 파일 5개를 알려 주세요
@vscode	편집기 기능, 설정, 확장 API	@vscode 자동 줄바꿈은 어떻게 켜나요?

으로 대상을 지정하는 법

를 치면 파일과 폴더, 심볼은 물론 Tool까지 목록에 함께 뜹니다. 이름을 외워 칠 필요가 없습니다.

참조	들어가는 내용	입력 예시
파일 이름	고른 파일의 내용 전체.	#quote.js 의 세금 계산 부분을 설명해 주세요
#selection	지금 편집기에서 선택한 범위. 선택한 것이 있을 때만 씁니다.	#selection 이 조건문을 표로 바꿔 주세요
#problems	화면에 떠 있는 오류와 경고.	#problems 무엇부터 고쳐야 하나요?
#codebase	저장소 전체. 어느 파일인지 모를 때 씁니다.	#codebase 탁송료를 계산하는 코드가 어디 있나요?
#runSubagent	Sub Agent에게 일을 위임합니다. 결과만 반환되어 Main Agent의 Context가 덜 무거워집니다.	#runSubagent quote.js 에 박힌 숫자를 전부 찾아 목록만 알려 주세요

※ #codebase 는 편해 보이지만 **범위가 가장 넓습니다**. 고칠 파일을 이미 아는데도 이것을 쓰면, Agent가 저장소를 뒤지면서 **관계없는 파일의 코드 조각까지 Context에 함께 담습니다**.
답이 나오지지는 않고 토큰만 더 쓰게 됩니다.

자주 쓰는 / 명령

같은 요청을 매번 문장으로 쓰지 않아도 됩니다. / 뒤에 명령 이름을 적으면 해당 작업으로 전환됩니다.

명령	역할	입력 예시
<code>/explain</code>	코드 블록, 파일, 개념을 설명합니다.	<code>/explain #selection</code>
<code>/fix</code>	코드 문제와 컴파일, 린트 오류를 고칠 방법을 제안합니다.	<code>/fix #quote.js</code>
<code>/tests</code>	편집기에서 선택한 메서드와 함수의 테스트를 만듭니다. 기존 테스트 파일에 덧붙이거나 새 파일을 만듭니다.	<code>/tests #quote.js</code>
<code>/setupTests</code>	테스트 프레임워크 설정을 돕습니다. 추천 항목과 설정 절차를 함께 제시합니다.	<code>/setupTests</code>
<code>/init</code>	프로젝트 구조와 코딩 패턴을 분석해 지시문 파일 을 만들거나 갱신합니다.	<code>/init</code>
<code>/plan</code>	Plan Agent 로 전환해, 요구사항을 확인한 뒤 구현 계획을 세웁니다.	<code>/plan #hol1-requirement.md</code>
<code>/compact</code>	대화 Context를 요약해 압축합니다. 대화가 길어졌을 때 씁니다.	<code>/compact</code>
<code>/clear</code>	Chat에서 새 세션을 시작합니다.	<code>/clear</code>

※ 목록은 설치한 확장과 MCP 서버, 저장소의 Prompt 파일과 Agent Skill에 따라 달라집니다. 입력창에 / 만 쳐서 **지금 쓸 수 있는 것**을 확인하십시오.

Agent가 추측으로 채우는 자리

요청에 적지 않은 것은 Agent가 스스로 채웁니다. 무엇으로 채우는지 알면 무엇을 적어야 하는지도 정해집니다.

빈자리	추측	대응
대상	지금 편집 중인 파일의 이름과 선택한 텍스트가 자동으로 들어가고, Agent가 그 두 가지로 대상을 짐작합니다.	#로 파일을 붙입니다. "이 코드"처럼 가리키는 말은 쓰지 않습니다.
출력 형태	공개 코드에서 흔한 관행을 따릅니다. 테스트 프레임워크를 적지 않으면 이 저장소가 쓰는 Vitest 대신 Jest 문법이 나오기도 합니다.	입력과 기대 출력을 값으로 적습니다. 기존 테스트 파일을 붙이면 그 자체가 예시가 됩니다.
설명하지 않은 코드의 동작	모델이 학습한 적 없는 코드라서, 공개 코드에서 흔한 패턴과 주변 코드로 그 동작을 지어냅니다.	그 모듈의 파일을 #로 붙이고, 무엇을 하는 모듈인지 한 줄 덧붙입니다.
판단이 갈리는 지점	사용자에게 질문하지 않고 그럴듯한 쪽을 골라 그대로 진행할 수 있습니다.	확실하지 않은 부분은 고치지 말고 먼저 질문해 달라고 적습니다.

※ Ask는 활성 파일을 항상 넣고, Agent는 요청을 보고 넣을지 스스로 판단합니다. 활성 파일은 지금 커서가 있는 편집기 하나이고, 탭 바의 나머지 파일은 들어가지 않습니다.

나쁜 요청과 좋은 요청

agent picker에서 Ask 를 고르고 시작합니다. 같은 저장소에 범위가 다른 두 요청을 보내고 답을 비교합니다.

단계	내용
01	Chat에 #codebase 이 프로젝트 코드 좀 정리해 주세요 만 보냅니다. 답을 다 읽지 말고 어느 파일을 몇 개나 건드리겠다고 하는지만 셉니다.
02	대상을 파일 하나로 좁히고 무엇을 할지, 무엇을 지킬지, 언제 끝인지를 적어 다시 보냅니다.
03	두 답을 나란히 놓고 파일 수와 제안 수가 얼마나 줄었는지 셉니다. 지킬 것과 완료 조건이 답에 반영됐는지도 함께 봅니다.

01단계에 넣을 요청

#codebase 이 프로젝트 코드 좀 정리해 주세요

02단계에 넣을 요청

#quote.js 의 calculateQuote에서 세금 계산 부분을 별도 함수로 분리해 주세요.

함수 시그니처는 그대로 두고, npm test가 전부 통과해야 합니다

※ 01에는 대상, 할 일, 지킬 것, 완료 조건이 하나도 없습니다. 어느 파일을 건드릴지, "정리"가 무엇인지, 시그니처를 바꿔도 되는지, 어디까지 하면 끝인지를 전부 Agent가 대신 정합니다.
02에는 19페이지에서 본 네 가지가 모두 들어 있습니다.

IV

Context 설계: 지시문 파일과 Agent Skills

모델이 보는 것, Context

Context는 모델이 응답을 생성할 때 참조할 수 있는 정보 전체를 뜻합니다. 요청마다 아래 일곱 가지가 하나의 Prompt로 조립되며, 모델의 판단은 그 Prompt에 포함된 정보로 한정됩니다.

요청마다
직접 넣는 것

User message

지금 입력창에 쓴 문장입니다. 요청마다 새로 씁니다.

Explicit references

로 붙인 파일, 폴더, 심볼, 웹 문서입니다. 범위를 좁히는 가장 확실한 수단입니다.

자동으로
들어가는 것

System instructions

Agent의 동작을 정의하는 내장 지침입니다. VS Code가 넣는 것이라 우리가 작성하지 않습니다.

Implicit context

편집 중인 파일, 선택 영역, 표시된 오류, git 상태입니다. 따로 붙이지 않아도 편집기 상태가 그대로 전달됩니다.

Conversation history

이 세션에서 오간 메시지 전부입니다. 쌓이다가 Context Window가 다 차면 VS Code가 앞부분을 짧게 줄여 자리를 만듭니다.

Tool outputs

파일 읽기, 터미널 실행, 코드 검색의 결과입니다. 도구를 많이 쓸수록 분량이 쌓입니다.

한 번
적어 두는 것

Customizations

지시문 파일, Agent Skill, Custom agents입니다. Context Window가 다 차거나 `/compact` 로 앞부분을 줄인 뒤에도 지시문은 매 요청에 다시 들어갑니다.

※ Context Window는 한 요청에 담기는 정보의 총량입니다. 대화 기록은 세션을 넘어 이어지지 않으므로, 사내 규약이든 설계 배경이든 일곱 자리 중 하나에 들어가야 반영됩니다.

우리가 통제할 수 있는 Context 항목

Context 설계는 Agent에게 필요한 프로젝트 정보를 골라서 주는 체계적인 방법입니다.

Context를 이루는 일곱 가지 중 여섯을 우리가 설계할 수 있으며, 이번 챕터는 그중 Customizations를 다룹니다.

자리	통제 수단	다루는 곳
User message	요청마다 대상, 할 일, 지킬 것, 완료 조건을 문장으로 씁니다.	앞 챕터
Explicit references	# 로 파일, 폴더, 심볼을 직접 붙입니다.	앞 챕터
Implicit context	작업할 파일을 열어 두고 그 편집기에 커서를 둡니다. 커서가 있는 편집기 하나가 기준입니다.	별도 세션 없음
Customizations	지시문 파일과 Agent Skill로 저장소에 남깁니다. 항상 적용되는 지시문은 매 요청에 포함되고, Skill은 필요할 때 불러 옵니다.	이번 챕터
Conversation history	새 작업은 새 세션에서 시작하고, 같은 주제에서 길어지면 /compact 로 줄입니다.	Hands-On Lab 3
Tool outputs	지금 작업에 필요 없는 Tool을 꺼 둡니다.	Hands-On Lab 3

※ 목록에서 빠진 System instructions 하나는 우리가 작성하는 자리가 아닙니다. Agent의 동작을 정의하는 내장 지침이라 VS Code가 넣습니다.

지시문 파일 네 가지

지시문은 요청마다 다시 적지 않아도 되도록, 프로젝트의 규약을 저장소에 파일로 남겨 둔 것입니다. 종류가 넷인 것은 적용 시점과 담는 내용이 다르기 때문입니다.

파일	위치	적용 시점	내용
copilot-instructions.md	.github/	모든 Chat 요청에 자동	프로젝트 전체에 해당하는 규약
AGENTS.md	Workspace 루트	모든 Chat 요청에 자동	프로젝트 전체 규약. Copilot 외의 AI 도구도 읽습니다.
*.instructions.md	.github/instructions/	applyTo 그룹이 맞는 파일에서	파일 종류나 모듈에만 해당하는 규약
*.prompt.md	.github/prompts/	/ 로 부를 때만	자주 보내는 요청문 자체. 실행할 Role과 모델도 함께 지정합니다.

※ .github 는 폴더 이름일 뿐입니다. VS Code가 로컬 Workspace에서 파일을 찾아 읽습니다. 다만 네 파일 모두 Chat에서만 동작하고, 편집기 인라인 제안에는 반영되지 않습니다.

copilot-instructions.md

저장소마다 한 개만 존재하는 파일입니다. 모든 Chat 요청에 자동으로 포함되므로, 프로젝트 전체에 적용되는 규약만 적습니다.

항목	예
코딩 스타일과 네이밍	변수와 함수에 camelCase, 상수에 UPPER_SNAKE_CASE 를 쓴다
기술 스택과 선호 라이브러리	테스트는 Vitest로 작성한다
따를 아키텍처 패턴	금액 계산은 순수 함수로 두고 입출력과 섞지 않는다
보안 요구와 오류 처리	API 응답은 파싱 직후에 형식을 검사한다
문서화 기준	공개 함수에는 JSDoc 으로 반환값의 단위를 밝힌다

```
.github/copilot-instructions.md
```

```
# 견적 계산 서비스 코딩 표준
```

```
## 기술 스택
```

- 런타임은 Node.js 22, 모듈은 ESM 을 쓴다
- 테스트는 Vitest 로 작성하고 describe 와 it 을 쓴다

```
## 네이밍
```

- 변수와 함수는 camelCase, 상수는 UPPER_SNAKE_CASE 를 쓴다
- 함수 이름은 동사로 시작한다 (calculateQuote, formatCurrency)

```
## 구조와 문서화
```

- 금액 계산은 순수 함수로 두고 입출력과 섞지 않는다
- 공개 함수에는 JSDoc 으로 반환값의 단위를 밝힌다

※ 파일 맨 위에 --- 로 감싼 설정 블록을 **frontmatter** 라고 부릅니다. 이 파일은 항상 적용되므로 **frontmatter 없이 본문만** 씁니다.

AGENTS.md

Workspace 루트에 둡니다. 적용 방식은 `copilot-instructions.md` 와 같습니다.
다른 점은 Copilot 외의 AI 도구도 같은 파일을 읽는다는 것입니다.

항목	내용
위치	Workspace 루트. 하위 폴더에 두는 방식도 실험 단계로 지원합니다.
적용	모든 Chat 요청에 자동으로 들어갑니다. <code>copilot-instructions.md</code> 와 차이가 없습니다.
쓰는 이유	한 저장소에서 여러 AI 도구를 함께 쓸 때입니다. 도구마다 다른 파일을 만들지 않아도 됩니다.
우선순위	<code>copilot-instructions.md</code> 와 <code>AGENTS.md</code> 중 어느 쪽이 먼저라는 규칙이 없습니다. 같은 규칙을 양쪽에 적지 마십시오.

※ Claude 계열 도구는 같은 위치의 `CLAUDE.md` 를 읽습니다.

*.instructions.md

frontmatter의 applyTo 에 적은 글롭 패턴과 맞는 파일을 작업할 때만 적용됩니다. 파일 종류나 모듈마다 규약이 다를 때, 범위를 나눠 여러 개를 둡니다.

APPLYTO	적용 대상
test/*.js	test 폴더 바로 아래에 있는 JavaScript 파일
test/**/*.js	test 폴더 하위 전체에 있는 JavaScript 파일
src/**	src 폴더 하위 전체에 있는 모든 파일. 확장자를 가리지 않습니다.
**	저장소의 모든 파일. 사실상 모든 요청에 적용됩니다.

```
.github/instructions/test.instructions.md
---
applyTo: "test/**/*.js"
---
# 테스트 코드 규칙

- describe 에는 테스트 대상 함수 이름만 적는다
- it 에는 조건과 기대 결과를 한 문장으로 적는다
  describe('calculateQuote', () => {
    it('법인이면 할인이 적용된다', ...)
  })
- 기대값을 코드로 계산하지 말고 값 그대로 적는다
- 정상 경로만 만들지 말고 경계값과 예외도 덮는다
```

※ * 는 폴더 한 단계 안에서만, ** 는 하위 폴더를 몇 겹이든 건너뛸니다. .gitignore 와 같은 문법입니다. applyTo 를 적지 않으면 자동으로 적용되지 않고, 그 지시문 파일을 # 로 붙인 요청에서만 적용됩니다.

지시문 작성 원칙

지시문은 매 Chat 요청에 함께 들어갑니다. Context Window가 다 차면 VS Code가 앞선 대화를 요약해 자리를 만드는데, 지시문이 길수록 그 시점이 빨라지고 초반 내용이 먼저 사라집니다.

원칙

내용

문장 단위	규칙 하나를 한 문장으로 적고, 답을 내용이 돌아면 지시문도 둘로 나눕니다. "함수는 50줄을 넘기지 않고 테스트는 Vitest로 작성한다" → "함수는 50줄을 넘기지 않는다" / "테스트는 Vitest로 작성한다"
규칙의 근거	규칙이 왜 있는지 적으면 예외 상황에서 더 나은 판단을 합니다. "세울은 catalog.js 의 상수를 쓴다. 세법이 바뀔 때 고칠 곳이 한 군데여야 하기 때문이다"처럼 적습니다.
코드 예시	권장 패턴과 피할 패턴을 실제 코드로 제시합니다. Copilot은 추상적인 규칙보다 실제 코드를 더 정확히 따릅니다.
린터 중복 제외	들여쓰기, 세미콜론처럼 포매터가 이미 강제하는 것은 적지 않습니다. 당연하지 않은 규칙만 남깁니다.
주제별 파일 분리	언어나 작업마다 다른 규칙은 *.instructions.md 를 여러 개 두고 applyTo 로 붙입니다.
버전 관리	Workspace에 두고 commit해야 팀이 같은 규약을 씁니다. 개인 프로필에 두면 나에게만 적용됩니다.

※ 무엇보다 적을지 막히면 코드 리뷰에서 세 번 이상 지적한 규칙을 첫 줄로 적으십시오. 사람이 반복해 말해야 했던 것은 Copilot에게도 반복해 말해야 합니다.

지시문 파일이 여러 개일 때의 적용 규칙

`copilot-instructions.md`, `AGENTS.md`, `*.instructions.md` 는 한 저장소에 함께 있을 수 있습니다. 여기에 개인 지시문과 조직 지시문까지 겹치면 무엇이 보장되고 무엇이 보장되지 않는지 알아야 합니다.

01

여러 파일의 동시 적용

VS Code는 하나만 골라 쓰지 않고 적용 조건을 만족한 파일을 모두 모아 대화 Context에 넣습니다. `*.instructions.md` 는 `applyTo` 가 맞을 때만 들어갑니다.

02

스코프에만 있는 우선순위

개인 지시문(사용자 홈 폴더의 `.copilot/instructions`, `commit`하지 않습니다)이 저장소 지시문(`.github/copilot-instructions.md` 또는 `AGENTS.md`)보다 앞섭니다.
반면 같은 스코프 안의 파일끼리는 순서가 보장되지 않습니다.

03

한 규칙은 한 곳에만

같은 주제를 여러 파일에 적지 마십시오. 두 파일이 서로 다른 말을 하면 어느 쪽이 이길지 정해져 있지 않고, 나중에 어느 파일을 고쳐야 하는지도 찾기 어렵습니다.

*.prompt.md

copilot-instructions.md, AGENTS.md, *.instructions.md 와 성격이 다릅니다. 자동으로 적용되지 않고 슬래시로 이름을 불러야 실행됩니다.

반복해 보내는 요청을 파일로 굳혀 두는 자리입니다.

FRONTMATTER 역할

name	슬래시 뒤에 칠 이름을 정합니다. 적지 않으면 파일 이름을 그대로 씁니다.
description	슬래시 명령 목록에 표시할 한 줄 설명을 적습니다.
agent	실행할 Role을 고정합니다. ask, agent, plan 중에서 고릅니다.
model	실행할 모델을 지정합니다. 적지 않으면 지금 고른 모델을 씁니다.
tools	이 Prompt를 실행하는 동안 쓸 Tool을 지정합니다. 아래 예시의 vscode/askQuestions 는 Agent가 되물을 질문을 화면에 띄우는 내장 Tool입니다.

```
.github/prompts/add-endpoint.prompt.md
```

```
---
```

```
name: add-endpoint
```

```
description: 건적 API에 엔드포인트를 추가한다
```

```
agent: plan
```

```
tools: ['vscode/askQuestions']
```

```
---
```

```
${input:endpoint:POST /quotes} 엔드포인트를 추  
가하는 계획을 세워 주세요.
```

```
기존 라우팅 규약과 오류 처리 방식을 따릅니다.
```

※ `${input:이름:예시값}` 은 실행할 때 값을 받아 채우는 자리입니다. 대부분의 모델이 이 표기를 알아보고 값을 되웁니다. 확실히 묻게 하려면 tools 에 `vscode/askQuestions` 를 넣습니다.

Agent Skill

Agent Skill은 지시와 템플릿, 스크립트를 한 폴더에 묶어 두고 요청이 설명과 맞을 때만 꺼내 읽는 작업 묶음입니다. 릴리스 노트 작성이 여기 속합니다.

구성	내용
저장 위치	프로젝트는 <code>.github/skills/</code> , 개인은 홈 폴더의 <code>.copilot/skills/</code> 아래 Skill 이름과 같은 폴더를 만듭니다.
SKILL.md	폴더마다 하나씩 둡니다. frontmatter에 name 과 description 이 필수이고, 본문에 절차를 적습니다.
description	로드 조건입니다. 요청이 설명과 맞을 때만 불러 오므로, 무엇을 언제 쓰는지 적습니다.
함께 두는 파일	템플릿, 예제, 스크립트를 같은 폴더에 둡니다. SKILL.md 에 [양식] (<code>./template.md</code>) 처럼 링크를 걸어야 읽힙니다.
만드는 법	Chat에 <code>/skills</code> 로 Configure Skills 메뉴를 열고, <code>/create-skill</code> 로 초안을 만들 수 있습니다.

```
.github/skills/release-note/
SKILL.md
template.md
examples/v2.3.md
```

```
SKILL.md
---
name: release-note
description: 머지된 PR을 모아 사내 양식으로 정리
한다.
  릴리스 노트 작성을 요청받았을 때 사용한다.
---
# 릴리스 노트 작성
1. 지난 태그 이후 머지된 PR을 모은다
2. [양식](./template.md) 순서대로 채운다
3. 톤은 [지난 노트](./examples/v2.3.md)를 따른다
```

Custom Instructions와 Agent Skills

Custom Instructions와 Agent Skills는 모두 저장소나 개인 프로필에 두는 파일이고, 모두 Context에 들어갑니다. 아래는 두 기능의 차이점입니다.

기준	CUSTOM INSTRUCTIONS	AGENT SKILLS
로드 시점	항상 적용되는 지시문은 매 요청에 들어갑니다. *.instructions.md 는 applyTo 가 맞을 때만 들어갑니다.	description 이 요청과 맞을 때 본문이 로드됩니다 . 그 전에는 이름과 설명만 후보로 검토됩니다.
구성 단위	마크다운 파일 하나입니다. 다른 파일이나 URL을 링크로 가리킬 수는 있습니다.	폴더 하나 입니다. 지시문과 스크립트, 예제가 그 폴더 안에 함께 들어갑니다.
용도	늘 지켜야 하는 규칙입니다. 코딩 표준, 네이밍 규칙, 금지 사항처럼 매번 지켜야 하는 것을 적습니다.	가끔 하지만 절차가 고정된 작업 입니다. 릴리스 노트 작성이나 배포 절차가 이런 작업입니다.

※ 규칙을 전부 지시문에 몰아넣으면 매 요청이 무거워집니다. **가끔 쓰는 절차는 Skill로 빼면** 평소에는 Context에 들어오지 않습니다.

/init 과 지시문 파일

지시문 파일을 만들어 두면 다음 요청부터 설명이 줄어듭니다.
agent picker에서 **Agent** 를 고르고 시작합니다.

단계	내용
01	Chat 입력창에 /init 지시문은 한국어로 작성해 주세요 를 입력하고 Enter를 누릅니다. 저장소를 훑어 AGENTS.md 초안이 만들어집니다.
02	파일을 열어 Copilot이 코드만 보고 알아낸 것을 찾습니다. 테스트 프레임워크, 모듈 방식, 폴더 구조가 이미 적혀 있습니다.
03	이제 코드만 봐서는 알 수 없는 팀 규약을 아래 한 줄로 덧붙이고 저장합니다.
04	/clear 로 새 세션을 열고 # 로 warmup.js 를 붙여 아래 요청을 보냅니다. 주석이 들어 있는지 봅니다.

03단계 - AGENTS.md 에 추가

테스트

- 각 it 안에 // given, // when, // then 주석으로 단계를 나눈다

04단계에 넣을 요청

#warmup.js 의 테스트를 만들어 주세요

※ Vitest는 package.json 에 있으므로 /init 이 알아서 찾아 적습니다. 주석 규약은 코드 어디에도 없어 사람이 적어야만 들어갑니다.

/create-skill 과 릴리스 노트

Skill은 요청이 description 과 맞을 때만 불러 옵니다.
agent picker에서 **Agent** 를 고르고 시작합니다.

단계	내용
01	Chat 입력창에 아래 요청을 보냅니다. <code>.github/skills/release-note/</code> 폴더와 <code>SKILL.md</code> 가 만들어집니다.
02	<code>SKILL.md</code> 의 description 을 봅니다. 언제 쓰는 Skill인지 적혀 있어야 자동으로 불러 옵니다. 빠졌으면 한 문장 채웁니다.
03	같은 폴더에 <code>template.md</code> 를 만들어 <code>##</code> 한 줄 요약, <code>##</code> 새로 들어간 것, <code>##</code> 고친 것, <code>##</code> 확인이 필요한 것 네 제목만 적고, <code>SKILL.md</code> 본문에 [양식](./template.md) 링크를 겁니다.
04	<code>/clear</code> 로 새 세션을 열고 아래 요청을 보냅니다. 요청문에 양식이 없는데도 네 항목이 순서대로 채워집니다.

01단계에 넣을 요청

`/create-skill` 릴리스 노트를 사내 양식으로 쓰는 `skill`을 만들어 주세요. 이름은 `release-note` 입니다

04단계에 넣을 요청

`v1.1` 릴리스 노트를 작성해 주세요. 이번 변경은 법인 고객 할인 추가와 제주 탁송료 수정 두 건입니다



Hands-On Lab 1: Requirement → Plan → Code

Hands-On Lab 1 진행 순서

요구사항 문서 한 장에서 시작해 계획을 거쳐 코드까지 갑니다. 각 단계의 산출물을 다음 단계에서 그대로 재사용합니다. 01단계부터 차례로 안내합니다.

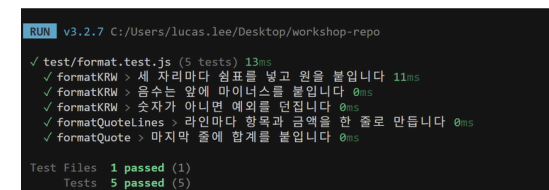
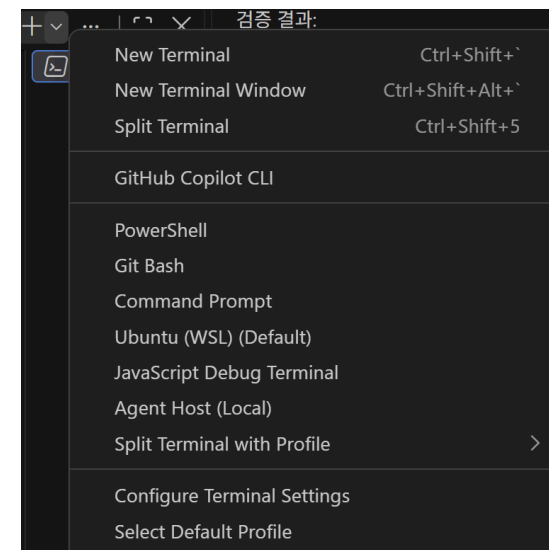
단계	하는 일	산출물
00	npm test 가 통과하는지 확인해 모두 같은 기준선에서 출발합니다.	통과하는 기준선
01, 02	앞서 만든 지시문 파일에 이번 과제에 필요한 규약을 덧붙입니다.	AGENTS.md
03	docs/hol1-requirement.md 를 붙여 /plan 으로 계획을 받습니다.	구현 계획
04	계획을 읽고 애매한 곳을 되물어 고칩니다.	확정된 계획
05	Start Implementation 으로 넘기고 결과를 검토합니다.	코드와 테스트

※ 계획은 세션 메모리에 저장되고 대화가 끝나면 사라집니다. 한 세션 안에서 끝내 주십시오.

시작 전에 확인할 네 가지

쓰던 환경을 그대로 이어 씁니다. 아래 네 가지만 지금 확인합니다.

확인	방법
터미널	터미널 패널의 드롭다운을 열어 쓸 수 있는 shell을 고릅니다. 목록은 설치 환경마다 다릅니다.
기준선	/init 실습에서 만든 test/warmup.test.js 가 있으면 먼저 지웁니다. 그다음 npm test 를 돌려 5개가 모두 통과하는지 봅니다.
지시문	앞서 만든 AGENTS.md 가 남아 있는지 봅니다. 없으면 01단계에서 새로 만듭니다.
모델	model picker에서 실습에 쓸 모델을 고릅니다. 실습 내내 바꾸지 않아야 결과가 같린 원인을 지시문과 계획에서 찾을 수 있습니다.



이 환경의 예시입니다

지시문 확인과 보강

/init 실습에서 만든 AGENTS.md 를 그대로 이어 씁니다. 이번 과제에 필요한 규약만 덧붙입니다.

확인

방법

파일 존재 없으면 Chat에 /init 지시문은 한국어로 작성해 주세요 를 보내 새로 만듭니다. /init 실습을 건너뛴 경우입니다.

앞서 넣은 줄 // given, // when, // then 주석 규약이 그대로 있는지 봅니다. 이번 과제의 테스트에도 그대로 적용됩니다.

프로젝트 파악 이 저장소를 무엇을 하는 코드라고 적었는지 봅니다. 틀렸다면 그 줄부터 고칩니다.

과제 규약 아래 두 줄을 코딩 규칙을 다루는 절 아래에 넣습니다. 코드만 봐서는 알 수 없어 사람이 넣어야 하는 규칙입니다.

분량 덧붙이기만 하고 기존 내용은 지우지 마십시오. Copilot이 알아낸 내용이 이번 구현에도 쓰입니다.

작업 규칙 아래에 넣을 두 줄

- 금액 계산에 매직 넘버를 쓰지 않는다. 이름 있는 상수로 뺀다
- 함수 하나가 한 가지 일만 하게 한다

※ 절 이름은 /init 이 만든 파일마다 다릅니다. 규칙을 모아 둔 절을 찾아 그 아래에 넣으십시오. 이 두 줄이 05단계 결과에서 그대로 드러납니다.

요구사항 문서와 /plan

요구사항 문서를 요약해 옮겨 적지 마십시오. 문서 자체를 붙여서 넘깁니다.

순서	내용
1	docs/hol1-requirement.md 를 열어 한 번 읽어 봅니다. 문서 끝의 "확인이 필요한 부분" 세 줄은 특히 눈여겨봅니다.
2	아래 요청을 입력창에 그대로 적어 보냅니다. 파일 이름은 # 를 입력해 목록에서 고릅니다.
3	picker가 Agent여도 /plan 을 치면 Plan agent로 전환됩니다. 코드는 아직 바뀌지 않습니다.

03단계에 넣을 요청

/plan #hol1-requirement.md 의 요구사항을 구현할 계획을 세워 주세요.

확인이 필요한 부분은 직접 정하지 말고 질문으로 남겨 주세요.

※ 계획이 나오기까지 30초 넘게 걸리기도 합니다. 기다리는 동안 받은 계획에서 확인할 네 가지를 읽어 두십시오.

받은 계획에서 확인할 네 가지

계획 대신 질문 목록이 먼저 올 수도 있습니다. 어느 쪽이든 아래 네 지점을 찾으십시오. 계획에 비어 있으면 되묻는 단계에서 문장으로 만들고, 질문으로 왔으면 그 자리에서 답합니다.

지점

확인 내용

대상 파일

새 모듈뿐 아니라 `src/demo.js` 까지 손델 파일 목록에 들어 있는지

없는 값

이자율처럼 요구사항에 없는 값을 임의로 가정하지 않고 질문으로 남겼는지

범위 밖 입력

선수금이 0~30%를 벗어날 때 오류를 던질지 보정할지 정해져 있는지

검증 방법

테스트를 어떻게 만들지가 들어 있는지. 없으면 구현 결과를 확인할 방법이 없습니다.

※ 요구사항 문서 끝에 **확인**이 필요한 항목 세 가지를 일부러 남겨 두었습니다. 계획이나 질문이 그중 몇 개를 짚었는지 세어 보십시오.

계획에 되물을 네 가지

계획은 한 번에 확정되지 않습니다. **답하면 또 물어 옵니다.** 문답이 두세 번 오가는 것이 정상입니다. 계획에서 찾은 빈 곳을 되묻는 문장으로 직접 만들어 보내십시오.

상황

보낼 말

혼자 정했을 때 계획이 이자율을 임의로 가정했다면 그 가정을 빼 달라고 말합니다.

범위가 넓을 때 기존 calculateQuote 를 고치려 하면 새 모듈로 분리해 달라고 말합니다.

검증이 없을 때 테스트 이야기가 없으면 어떻게 확인할지 계획에 넣어 달라고 말합니다.

탁송료를 물어올 때 계획이 제주 탁송료 불일치를 찾아 함께 고칠지 물어 오면 **아래 문장으로** 답합니다. 요구사항에 없던 문제라 이번 범위가 아닙니다.

탁송료를 물어오면 이렇게

제주 탁송료 불일치는 이번 범위가 아닙니다. 계획에서 빼 주세요.

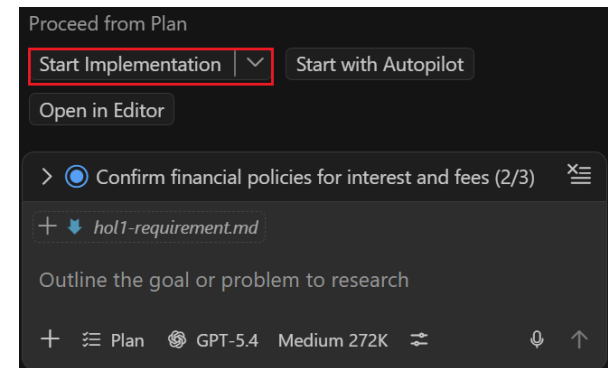
※ 질문이 끝나기를 기다리지 마십시오. 구현할 파일과 검증 방법이 정해졌으면 그만 물어보고 계획을 확정해 달라고 말하면 됩니다.

실행하고 결과 검토

여기서부터는 Agent가 일합니다. 사람이 할 일은 보고 판단하는 것입니다.

순서	내용
1	Start Implementation 을 누릅니다. 옆의 Start with Autopilot 은 승인도 묻지 않고 되묻는 질문에도 스스로 답하며 끝났다고 판단할 때까지 혼자 돕니다. 사람이 볼 자리가 없어 오늘은 쓰지 않습니다.
2	실행되는 동안 승인을 묻는 화면이 나올 수 있습니다. 무엇을 하려는지 보고 진행합니다.
3	끝나면 바뀐 파일 목록을 먼저 봅니다. 계획에 없던 파일이 있으면 먼저 열어 확인합니다.
4	diff를 훑어 계획대로 고쳤는지 봅니다. Agent가 테스트를 돌리지 않고 끝냈다면 <code>npm test</code> 를 직접 돌립니다.
5	결과가 계획과 다르면 체크포인트 복원으로 되돌립니다. 계획 단계로 돌아가 요청을 고쳐 다시 시작할 수도 있습니다.

※ 3번과 4번을 건너뛰고 바로 테스트 결과부터 보는 분이 많습니다. 무엇이 바뀌었는지 모르는 채 통과한 테스트는 아무것도 보증하지 않습니다.



계획 아래에 나타납니다

완료 체크리스트

아래 여섯 항목이 전부 충족되면 완료입니다. 코드 스타일은 보지 않습니다.

항목	기준
기능	계약 기간과 선수금을 받아 월 납입금을 돌려주는 함수가 생겼습니다.
상수	36, 48, 60개월의 잔존 가치 비율이 이름 있는 상수로 들어가 있습니다.
예외	선수금이 0~30%를 벗어난 입력에 대한 처리가 있습니다.
회귀	기존 npm test 가 여전히 전부 통과합니다.
검증	새 기능의 테스트가 최소 하나 있습니다.
실행	npm start 를 돌리면 견적에 이어 월 납입금이 출력됩니다. src/demo.js 도 함께 고쳐졌는지 봅니다.

지시문이 만든 차이

같은 요구사항 문서에서 출발했는데 결과물이 갈립니다. 무엇이 그 차이를 만들었는지를 확인하는 자리입니다.

항목

내용

결과 비교

두세 분의 화면을 나란히 띄웁니다. 지시문에 적은 내용에 따라 테스트 스타일과 함수 분리가 갈립니다.

차이의 원인

같은 모델을 썼는데도 결과가 다릅니다. 갈린 것은 지시문에 무엇을 적었는지와 계획을 어디까지 되물었는지입니다.

실행 결과

npm start 로 견적을 출력해 봅니다. 월 납입금까지 나오는지, 금액이 서로 같은지 화면에서 바로 비교됩니다.

남은 질문

구현이 덜 끝났어도 괜찮습니다. 어디서 막혔는지 이야기해 주시면 함께 살펴보겠습니다.

VI

Hands-On Lab 2: Refactoring, Testing & Debugging

순서가 바꾸는 결과

같은 코드를 같은 Agent에게 맡겨도 테스트를 언제 만드느냐에 따라 결과가 갈립니다.
고치기 전에 출력값을 기록해 두면, 고친 뒤 값이 달라졌을 때 바로 알 수 있습니다.

단계	테스트 나중	테스트 먼저
첫 요청	이 파일 Refactoring 해 주세요	고치기 전에 여러 입력에 대한 출력값을 기록해 두는 테스트를 먼저 만들게 합니다.
구조 변경	한 번에 전부 바꿉니다.	한 단계씩 바꾸고 매번 <code>npm test</code> 를 돌립니다.
결과 판단	그렇듯해 보이면 받아들입니다.	테스트 통과 여부로 판단합니다.
동작이 바뀌면	알아채지 못한 채 넘어갑니다.	출력값이 달라지는 순간 동작이 바뀌었다는 사실을 알 수 있으므로 그 변경만 되돌립니다.

※ 테스트를 먼저 만드는 쪽이 느려 보이지만 되돌릴 지점이 남습니다. 그렇지 않은 쪽은 어디서부터 잘못됐는지 찾는 데 시간을 씁니다.

quote.js에 남겨둔 것

오늘 고칠 대상은 calculateQuote 함수 하나입니다. 아래 네 가지가 얹혀 복잡도를 높입니다.

구분	해당 행	내용
한 함수에 전부	15~111	검증, 옵션, 세금, 탁송료, 할인, 차감을 함수 하나가 혼자 다 합니다.
이어지는 계산	29~104	앞 단계가 더한 total 을 뒤 단계가 그대로 받습니다. 한 부분만 떼어 내기 어렵습니다.
박힌 숫자	60~62, 71~79	세율과 탁송료가 코드에 숫자로 적혀 있습니다. catalog.js 에 표가 있는데도 쓰지 않습니다.
비슷한 코드	68~82, 87~97	탁송료 조건문이 3단으로 겹쳐 있고, 법인 할인 두 갈래가 할인율만 빼고 같습니다.

Hands-On Lab 2 진행 순서

코드를 고치기 전에 테스트를 먼저 만듭니다. 01단계부터 차례로 안내합니다.

단계	하는 일	산출물
01	Agent에게 calculateQuote 를 읽고 구조를 고쳐야 할 곳을 찾게 합니다	고칠 목록
02	지금 출력값을 기록해 두는 테스트를 만들고 전부 통과하는지 확인합니다	test/quote.test.js
03	상수 분리, 표 사용, 함수 분리, 중복 제거를 한 번에 하나씩 요청합니다	고쳐진 quote.js
04	계산 결과가 달라지는 변경을 만나면 진행할지 사람이 정합니다	판단 기록

※ 이 실습은 구조를 바꾸는 기계적인 작업이라 판단이 많이 필요하지 않습니다. 가벼운 모델로 바꿔 진행하십시오. 결과는 테스트가 판정하므로 모델을 낮춰도 안전합니다.

수정이 필요한 영역 파악

Agent에게 읽고 수정이 필요한 부분을 확인합니다.

이 단계에서 볼 것은 구조뿐입니다. 네이밍이나 주석은 오늘 대상이 아닙니다.

순서	내용
1	agent picker를 Ask 로 두고 아래 요청을 보냅니다. #codebase 를 붙여 저장소 전체를 보게 합니다.
2	Agent의 답변을 확인합니다.
3	49페이지에 적어 둔 quote.js 문제 네 가지와 비교합니다. 표현이 달라도 같은 곳을 가리키는지 봅니다.

01단계에 넣을 요청

#codebase #quote.js 의 calculateQuote 에서 구조를 고쳐야 할
곳을 찾아 주세요. 저장소에 이미 있는 것을 다시 만들고 있지는
않은지도 함께 봐 주세요. 네이밍과 주석은 이번 대상이 아닙니다.

※ Ask 로 두면 파일이 바뀌지 않습니다. 아직 고칠 때가 아닙니다. 무엇을 고칠지 정하는 것이 이 단계의 전부입니다.

회귀 테스트 생성

코드를 고치기 전에 **지금 출력값을 기록해 두는 테스트**를 만듭니다.

이 테스트가 전부 통과해야 다음 단계로 갑니다. **실패하면 테스트가 잘못 만들어진 것입니다.**

순서	내용
1	agent picker를 Agent 로 바꿉니다. 이제 파일을 만들어야 합니다.
2	아래 요청을 보냅니다. test/quote.test.js 가 만들어집니다.
3	만들어진 파일을 열어 기대값이 숫자로 적혀 있는지 봅니다. 계산식으로 적혀 있으면 다시 시킵니다.

02단계에 넣을 요청

/tests #quote.js 의 calculateQuote 가 지금 내놓는 출력값을 그대로 기록하는 테스트를 만들어 주세요. 기대값은 실제 실행 결과로 채우고, 지역과 고객 유형, 옵션과 패키지 조합을 골고루 덮어 주세요.

※ 이 테스트는 **코드가 옳다고 보증하지 않습니다.** 버그가 있으면 버그까지 그대로 기록됩니다. 지금 값을 붙잡아 두는 것이 목적입니다.

Refactoring & Debugging

네 가지를 한 번에 시키지 마십시오. 아래 순서대로 한 번에 하나씩 요청하고, 네 요청 모두에 아래 두 줄을 붙입니다.

순서

보낼 요청

- 1 코드에 박힌 숫자를 이름 있는 상수로 빼 주세요.
- 2 택송료 계산이 `catalog.js` 의 표를 쓰도록 바꿔 주세요.
- 3 세금 계산을 별도 함수로 분리해 주세요.
- 4 법인 할인 두 갈래를 하나로 합쳐 주세요.

1번을 보낼 때의 전체 문장

`#quote.js` 의 `calculateQuote` 에서 코드에 박힌 숫자를 이름 있는 상수로 빼 주세요. 다른 것은 바꾸지 마세요.
계산 결과가 달라지면 진행하지 말고, 무엇이 얼마나 달라지는지 알려 준 뒤 제 판단을 기다려 주세요.

※ 뒤의 두 줄이 없으면 Agent가 기대값을 고쳐서라도 통과시키려 합니다. 2번에서 실제로 그런 상황이 생깁니다.

동작 변경 판단

2번 요청에서 Agent가 멈추고 물어 옵니다. 표를 쓰면 제주 금액이 달라지기 때문입니다. 진행할지 말지 정하는 것은 사람입니다.

순서	내용
1	Agent가 알려 준 달라지는 금액을 읽습니다. 제주 택송료가 40,000원 오릅니다.
2	표가 최신인지 코드가 최신인지 알 수 있는 근거가 없습니다.
3	근거가 없으므로 이 변경은 하지 않습니다. Agent에게 진행하지 말라고 답합니다.
4	나머지 요청은 그대로 진행합니다. 남긴 문제는 따로 적어 둡니다.

※ 구조를 고치는 일과 값을 정하는 일은 다릅니다. 근거 없이 값을 바꾸면 Refactoring이 아니라 동작 변경입니다.

완료 체크리스트

아래 네 항목이 모두 맞으면 완료입니다.

항목	기준
상수	코드에 박혀 있던 숫자가 이름 있는 상수로 바졌습니다.
함수	세금 계산이 별도 함수로 빠졌고, 법인 할인 두 갈래가 하나로 합쳐졌습니다.
테스트	02단계에서 만든 테스트가 전부 통과합니다.
실행	npm start 합계가 시작할 때와 같습니다. 구조만 바꿨으므로 금액은 달라지지 않습니다.

※ 제주 택승료는 고치지 않고 남겨 두었습니다. 네 가지 중 하나를 못 한 것이 아니라, 근거가 없어 하지 않기로 정한 것입니다.

Agent에게 Refactoring을 맡길 때

Agent에게 Refactoring을 맡길 때 아래 내용들을 신경써야 합니다.

항목	내용
요청의 크기	한 번에 하나씩 시켜야 diff를 끝까지 읽을 수 있습니다. 네 가지를 한 번에 시키면 무엇이 바뀌었는지 확인하지 못합니다.
바뀌지 않았다는 증거	테스트가 없으면 동작이 달라져도 모릅니다. 구조만 바꿨다는 것을 증명할 수단이 곧 회귀 테스트입니다.
사람의 판단	구조를 바꾸는 일은 Agent가 합니다. 결과가 달라지는 결정은 사람이 합니다. 요청에 그 조건을 적어 두어야 Agent가 혼자 정하지 않습니다.

VII

Hands-On Lab 3: Context & Token 효율화

사용량을 보는 곳

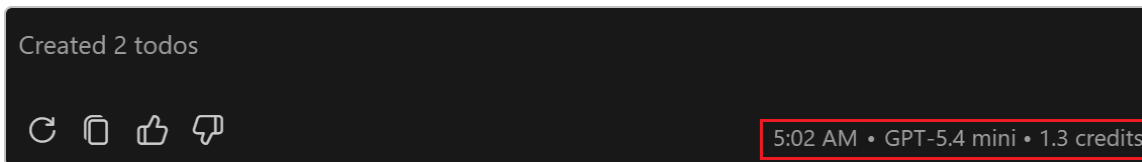
Credit과 Context 사용량은 두 곳에서 읽습니다. 실습하는 내내 들여다볼 화면이므로 지금 각자 찾아 두십시오.

읽는 값

읽는 위치

요청 하나의 비용

답변 바로 아래에 쓴 모델 이름과 Credit이 적힙니다.

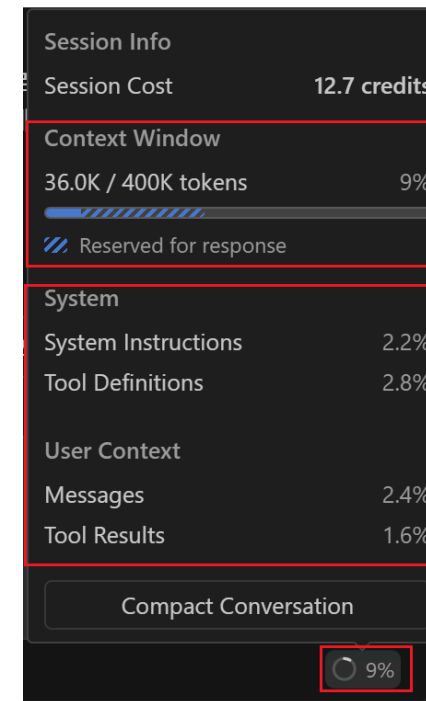


세션 누적 비용

입력창 우측의 백분율 배지를 누르면 팝오버가 열리고, 맨 위 Session Cost 에 이 세션에서 지금까지 쓴 Credit이 나옵니다.

Context Window 사용량

같은 팝오버에서 System Instructions, Tool Definitions, Messages, Tool Results 가 각각 몇 퍼센트를 차지하는지 갈라 봅니다.



세션 누적 12.7 Credit

※ 다음 실습의 00단계에서 적어 둘 시작 숫자가 Session Cost 값입니다. 요청마다 표시되는 숫자를 손으로 더하지 마시고 이 값을 그대로 읽으십시오.

대화가 길어지면 생기는 일

Context는 요청마다 새로 시작하지 않습니다. Session Cost 팝오버에 나오던 항목들이 이렇게 쌓입니다.

01 주고받은 대화가 계속 따라옵니다

팝오버의 Messages 항목입니다. 앞서 주고받은 내용이 매 요청에 다시 들어가므로, 열 번째 질문은 첫 번째 질문보다 무겁습니다.

02 Tool을 부른 결과도 쌓입니다

팝오버의 Tool Results 항목입니다. 파일을 읽거나 검색한 결과가 그대로 남습니다. 부를 일이 없는 Tool을 꺼 두면 Tool Definitions 와 Tool Results 가 함께 줄어듭니다.

03 Context Window가 다 차면 앞부분이 요약됩니다

Context Window가 다 차면 오류로 멈추지 않고 VS Code가 앞선 대화를 요약해 자리를 만듭니다. 이때 대화 초반에 말해 둔 조건이 뭉개집니다. 팝오버의 Compact Conversation 이나 /compact 로 직접 압축할 수도 있습니다.

※ 여기서 말하는 조건은 "계산 로직은 건드리지 말라", "상수는 catalog.js에만 두라" 처럼 대화 도중에 말로 걸어 둔 제약입니다. 이런 조건은 요약되면서 사라지지만, 지시문 파일에 적어 둔 규칙은 매 요청에 다시 들어갑니다.

Hands-On Lab 3 진행 순서

같은 과제를 조건만 바꿔 두 번 실행하고 숫자를 비교합니다.
01단계부터 차례로 안내합니다.

단계	하는 일	산출물
00	새 세션을 열어 앞 실습의 대화를 건너뛸니다	깨끗한 출발점
01	쓰던 조건 그대로 과제를 한 번 실행합니다	1회차 Credit과 Context 내역
02	세션을 다시 열고 Tool을 줄여 같은 과제 를 실행합니다	2회차 Credit과 Context 내역
03	두 회차를 비교해 무엇이 줄었는지 정리합니다	비교 결과

※ 두 회차의 과제가 **완전히 같아야** 비교가 성립합니다. 요청 문장을 한 글자도 고치지 마시고, 모델도 그대로 두십시오.

1회차 — Tool을 전부 컨 채로

앞 실습의 대화를 건너낸 깨끗한 세션에서 한 번 실행해 비교 기준을 만듭니다.
과제는 `quote.js` 를 통째로 읽어와 답할 수 있는 것으로 고정합니다.

순서

하는 일

- | | |
|----|---|
| 00 | /clear 로 새 채팅 세션을 엽니다. Chat view의 New Chat 버튼도 같은 일을 합니다. 앞 실습의 대화가 따라오면 1회차 숫자가 흐려집니다. |
| 01 | 쓰던 모델을 그대로 두고, Tool을 전부 컨 상태로 아래 요청을 보냅니다. |
| 02 | 답이 끝나면 팝오버를 열어 Session Cost 와 네 항목 백분율을 1회차 기록으로 적습니다. |

두 회차에 똑같이 넣을 요청

#quote.js 에 숫자로 박혀 있는 값을 전부 찾아
어떤 계산에 쓰이는지 표로 정리해 주세요.

※ 답변에 표가 나오고 숫자가 열 개 남짓 잡히면 정상입니다. 이 답의 내용을 2회차와 비교하게 되므로 화면을 닫지 마십시오.

2회차 — Tool을 줄인 채로

세션도 모델도 1회차와 똑같이 두고 Tool 구성만 줄여 같은 요청을 다시 보냅니다.

순서

하는 일

- | | |
|----|--|
| 00 | /clear 로 새 채팅 세션을 다시 엽니다. 1회차와 같은 출발점을 만드는 것입니다. |
| 01 | 입력창의 Configure Tools 를 열어 아래 Tool의 체크를 풉니다. 1회차와 달라지는 점은 Tool 구성 하나뿐입니다. |
| 02 | 1회차와 똑같은 요청을 그대로 붙여 넣고 보냅니다. |
| 03 | 답이 끝나면 Session Cost 와 네 항목 백분율을 2회차 기록으로 적습니다. |

2회차에 꺼 둘 Tool

```
web  vscode/extensions  vscode/installExtension
vscode/runCommand  vscode/VSCodeAPI  vscode/getProjectSetupInfo
runInTerminal  todos  memory
```

※ 목록에 없는 Tool을 더 꺼도 됩니다. 다만 readFile 과 codebase 는 이 과제에 필요하므로 켜 두어야 합니다.

두 회차의 차이

Credit 총액만 보서는 남는 것이 없습니다. 어느 항목이 줄어 그 차이가 났는지까지 확인해야 다음 작업에 쓸 수 있습니다.

비교 항목	읽는 곳	확인 내용
이번 회차에 쓴 Credit	답이 끝난 뒤의 Session Cost	두 회차 모두 새 세션에서 요청 하나만 보냈으므로 값을 그대로 비교합니다.
Tool Definitions	팝오버의 System 영역	Tool을 켜므로 2회차가 낮아야 정상입니다.
Tool Results	팝오버의 User Context 영역	부를 Tool이 줄었으니 결과도 줄어든 수 있습니다. 애초에 안 부르던 Tool만 켜다면 그대로입니다.
Messages	팝오버의 User Context 영역	두 회차 모두 새 세션에서 시작했으므로, 차이가 났다면 주고받은 횟수가 달랐다는 뜻입니다.
답의 품질	두 답변을 나란히 놓고 읽습니다	찾아낸 숫자의 개수와 설명이 1회차와 같은지 봅니다.

※ 줄지 않거나 오히려 늘 수도 있습니다. 같은 요청이라도 Agent가 부르는 Tool과 읽는 파일이 매번 똑같지는 않기 때문이며, 그것을 확인하는 것도 이 실습의 결과입니다.

현업에서 쓰는 다섯 가지

일반 사용자는 아래 내용들을 통해 Context와 Credit을 줄일 수 있습니다. 앞의 두 줄은 방금 실습에서 직접 써 봤습니다.

수단	줄어드는 대상	쓰는 때
새 세션	Messages	주제가 바뀌었을 때. 앞 대화를 끌고 갈 이유가 없으면 /clear 가 가장 값싼 수단입니다.
Tool 축소	Tool Definitions, Tool Results	쓸 Tool이 정해진 작업일 때. 답이 나빠지면 너무 많이 끈 것입니다.
# 로 대상 지정	Tool Results	고칠 파일을 알 때. #codebase 로 찾게 하면 검색 결과가 그대로 Context에 남습니다. 파일을 직접 붙이면 검색 자체가 일어나지 않습니다.
압축	Messages	같은 주제에서 대화가 길어졌을 때. Compact Conversation 또는 /compact 입니다.
모델 변경	토큰당 단가	기계적인 변환이나 형식만 바꾸는 작업에는 mini 등 가벼운 모델을 씁니다. 토큰 수가 그대로여도 단가가 내려가 Credit이 줄어듭니다. 다만 대화 중간이 아니라 새 세션에서 고릅니다.

※ 오래 지켜야 할 조건은 대화에 말하지 말고 지시문 파일에 적으십시오. 압축돼도 사라지지 않고, 매 요청에 다시 들어갑니다.

VIII

MCP 개념 & 사내 시스템 연계 방안

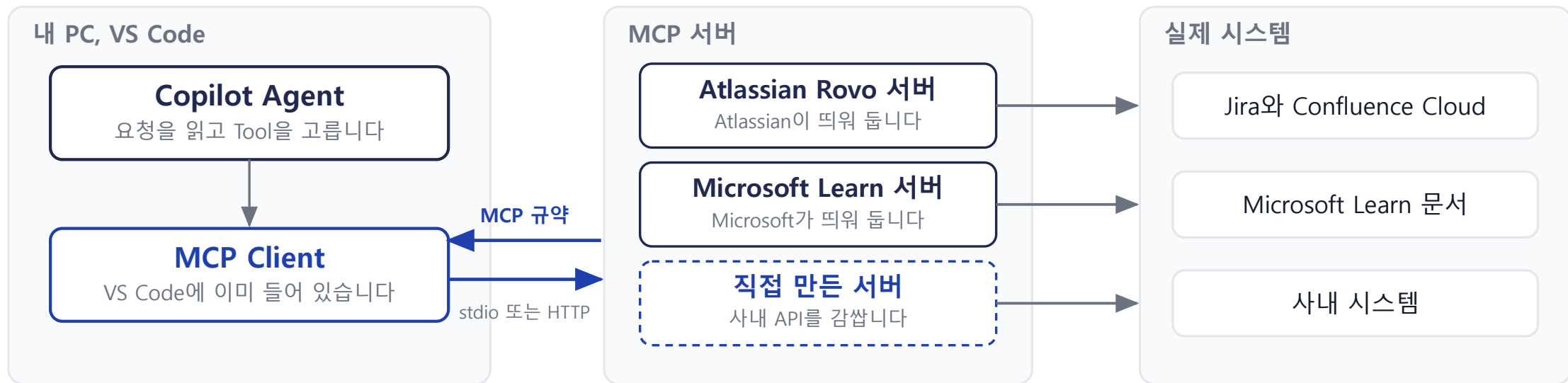
MCP는 무엇을 잇는가

MCP는 Agent가 외부 시스템의 기능을 Tool로 쓸 수 있게 하는 규약입니다.

1 Agent가 부를 Tool을 고릅니다

2 서버가 제공하는 Tool이 목록에 들어옵니다

3 서버가 시스템 API를 부릅니다



VS Code에 적는 것은 MCP 서버의 주소나 실행 명령뿐입니다. 실제 시스템과 주고받는 일은 MCP 서버가 맡습니다.

MCP 서버를 붙이면 달라지는 것

등록한 서버의 Tool이 Agent의 Tool 목록에 그대로 들어옵니다. 지금까지 쓰던 내장 Tool 옆에 나란히 놓입니다.

항목	내용
등록 위치	Workspace의 <code>.vscode/mcp.json</code> 에 서버를 적습니다. <code>mcp.json</code> 을 commit하면 팀이 같은 설정을 씁니다.
연결 방식	원격 서버는 주소로, 로컬 서버는 실행 명령으로 등록합니다. 어느 쪽이든 <code>type</code> 을 함께 적습니다.
등록한 뒤	Configure Tools 목록에 그 서버의 Tool이 나타나고, Agent가 요청에 맞춰 골라 씁니다. 쓰지 않을 Tool은 여기서 꺼 둡니다.
달라지는 것	Copilot이 우리 코드만 보는 것이 아니라 업무 시스템의 데이터를 읽고 씁니다. 이슈 조회, 문서 검색, 사내 API 호출이 요청 한 문장으로 이어집니다.

※ Tool이 늘어난 만큼 Tool Definitions가 Context Window를 더 차지합니다. 앞 실습에서 본 그대로이므로, 서버를 붙였다고 모든 Tool을 켜 둘 이유는 없습니다.

MCP 등록 방법

VS Code가 읽는 자리는 둘이고, Agent Host가 따로 읽는 자리가 하나 더 있습니다. 어디에 두느냐에 따라 적용 범위가 갈립니다.

두는 곳	파일	적용 범위
Workspace	저장소 루트의 <code>.vscode/mcp.json</code>	그 저장소를 연 사람 모두. 파일을 commit하면 팀이 같은 서버 목록을 씁니다.
개인	명령 팔레트에서 MCP: Open User Configuration 으로 엽니다	내 VS Code 전체. 저장소를 가리지 않고 쓰는 개인용 서버를 둡니다.
Agent Host 검용	저장소 루트의 <code>.mcp.json</code> 또는 <code>~/.copilot/mcp-config.json</code>	Agent Host가 직접 읽는 자리입니다. Copilot harness 세션과 Copilot CLI가 같은 설정을 쓰게 할 때 여기에 둡니다.

※ 파일을 저장한 뒤 **서버를 시작해야** Tool이 잡힙니다. 토큰이나 API 키 같은 비밀 값은 파일에 적지 말고 inputs 에 선언한 뒤 `${input:id}` 로 받아 쓰십시오. 다만 `${input:id}` 를 쓴 서버는 **Agent Host로 전달되지 않으므로**, harness를 Copilot으로 두고 쓸 서버는 `.mcp.json` 에 적습니다.

원격 서버와 로컬 서버

갈리는 기준은 MCP 서버가 어디서 도는지입니다. 연결할 시스템이 사내에 있는지와는 별개입니다.

기준	로컬 서버	원격 서버
도는 곳	내 PC에서 프로세스로 돛니다. VS Code가 실행 명령으로 띄웁니다.	다른 서버에서 이미 돌고 있고, 주소로 붙습니다.
실행 환경	npx 나 uvx 로 띄우므로 Node.js나 Python이 PC에 있어야 합니다.	따로 필요 없습니다. 그 주소로 나갈 수만 있으면 됩니다.
자격 증명	env 나 envFile 로 넘깁니다.	headers 나 oauth 로 처리하며, 브라우저 로그인을 쓰는 경우가 많습니다.
여러 사람이 쓸 때	각자 PC에서 따로 돛니다. 실행 환경과 버전이 사람마다 갈릴 수 있습니다.	서버 한 곳만 관리하면 됩니다.

※ 사내 시스템이라고 로컬 서버인 것이 아닙니다. 사내망의 Jira를 붙이더라도 그 앞에 세울 MCP 서버를 사내 서버에 올려 두면 팀 전체가 원격으로 씁니다.

원격 서버와 로컬 서버

`type` 이 둘을 가릅니다. 원격은 주소를, 로컬은 실행 명령을 적습니다.

구분	TYPE	필수 항목	선택 항목
원격 서버	http	url	headers 로 인증 헤더, oauth 로 인증 설정
로컬 서버	stdio	command	args, env, envFile, cwd

원격 서버 – 주소만 적으면 됩니다

```
{ "servers": { "github": {
  "type": "http",
  "url": "https://api.githubcopilot.com/mcp" } } }
```

로컬 서버 – VS Code가 이 명령으로 프로세스를 띄웁니다

```
{ "servers": { "playwright": {
  "type": "stdio",
  "command": "npx",
  "args": ["@playwright/mcp@latest"] } } }
```

※ sse 라는 값도 받지만 구버전 전송 방식입니다. MCP 규격에서 Streamable HTTP로 대체됐고 VS Code도 HTTP를 먼저 시도한 뒤 안 될 때만 물러섭니다. 새로 붙이는 서버에는 http 를 쓰십시오.

Microsoft Learn MCP 서버 연동

Microsoft Learn 서버는 인증도 요금도 없이 주소만 적으면 붙습니다.

순서	하는 일
01	Workspace에 <code>.vscode/mcp.json</code> 을 만들고 아래 내용을 적습니다.
02	편집기에 뜨는 inline action 으로 서버를 시작합니다. 상태가 Running 이 되어야 다음으로 넘어갑니다.
03	Configure Tools에서 <code>microsoft-learn</code> 을 체크하고 내장 web, fetch 는 끕니다. 켜 두면 Agent가 MCP 대신 그쪽을 부릅니다.
04	아래 요청을 보내고, 답변 위에 MCP: microsoft-learn 줄이 뜨는지 봅니다.

```
.vscode/mcp.json
{ "servers": { "microsoft-learn": {
  "type": "http", "url": "https://learn.microsoft.com/api/mcp" } } }
```

확인용 요청

Azure Functions의 Node.js 지원 버전을 공식 문서에서 찾아
출처 주소와 함께 알려 주세요.

※ 그래도 다른 Tool을 부르면 요청 앞에 `#microsoft_docs_search` 를 붙여 쓸 Tool을 직접 지정하십시오.

Jira 및 Confluence 연동

사내 Jira와 Collab(Confluence)은 Data Center라 Atlassian이 띄워 둔 원격 서버로는 붙지 않습니다. 오픈소스 **mcp-atlassian** 서버를 내 PC에서 띄워 붙입니다.

우리 환경

붙이는 방법

Data Center

mcp-atlassian 을 uvx 로 띄우고, 사내 주소와 개인 토큰을 env 에 넣습니다. 사전 준비에서 Python과 uvx 를 설치한 이유가 여기에 있습니다.

Cloud

Atlassian의 Rovo MCP Server 주소만 적고 브라우저로 로그인하면 됩니다. 설치할 것이 없습니다.

.vscode/mcp.json - 토큰은 사전 준비 자료에서 발급한 값을 넣습니다

```
{ "servers": { "mcp-atlassian": {
  "type": "stdio", "command": "uvx", "args": ["mcp-atlassian"],
  "env": { "JIRA_URL": "...", "JIRA_USERNAME": "...", "JIRA_PERSONAL_TOKEN": "...",
    "CONFLUENCE_URL": "...", "CONFLUENCE_USERNAME": "...", "CONFLUENCE_PERSONAL_TOKEN": "..." } } } }
```

붙었는지 먼저 확인합니다

현재 사용 가능한 MCP 도구 목록을 알려줘.

Jira 또는 Confluence 관련 도구가 연결되어 있는지도 확인해줘.

MCP 사내 적용 전 검토 사항

검토 항목

확인할 내용

서버 존재 여부 그 제품이 MCP 서버를 제공하는지, 제공한다면 우리가 쓰는 배포 형태까지 지원하는지

인증 브라우저 인증인지 토큰인지. 사용자마다 권한이 다르게 적용되는지

승인 절차 새 MCP 서버를 붙이려면 누구의 승인이 필요한지, 허용 목록을 누가 관리하는지

네트워크 사내에서 그 서버 주소로 나갈 수 있는지. 원격 연결은 여기서 가장 자주 막힙니다

라이선스 그 제품의 어느 요금제부터 쓸 수 있는지, 호출 수 제한이 걸리는지

※ 첫 줄에서 걸리는 경우가 가장 많습니다. 제품이 서버를 주지 않으면 직접 만드는 쪽으로 방향이 정해집니다.



Capstone Hands-On Lab

두 번 잡히는 옵션

테크 패키지에는 헤드업 디스플레이가 이미 들어 있습니다. 헤드업을 개별 옵션으로 한 번 더 고르면 같은 980,000원이 두 번 더해지고, 견적서에도 두 줄로 찍힙니다.

catalog.js - 테크 패키지에 든 옵션

```
tech: { name: '테크', includes: ['hud', 'premium-audio'], discountRate: 0.15 }
```

quote.js 29~55행 - 두 루프가 서로를 모릅니다

```
29 // 개별 옵션
35     total = total + opt.price;           // 고른 hud 980,000원이 더해집니다
40 // 패키지
48     const inner = OPTIONS[pkg.includes[j]]; // tech.includes 에도 hud 가 있습니다
50     sum = sum + inner.price;             // 같은 980,000원이 또 더해집니다
```

주문

합계

견적서 항목

demo.js 기본값 그대로

41,669,149원

테크 패키지 1,836,000

options 에 'hud' 추가

42,817,219원

헤드업 디스플레이 980,000, 테크 패키지 1,836,000

※ 차액 1,148,070원은 헤드업 값 980,000원과 거기에 붙은 세금입니다. 겹친 옵션을 한 번만 세도록 고치는 것이 오늘 마지막 과제입니다.

오늘 배운 것을 한 번에

네 단계가 각각 오늘의 한 챕터에 대응합니다. 순서를 지키는 것이 이 실습의 요점입니다.

단계	하는 일	도구
01	문제를 드러내는 테스트를 먼저 씁니다. 패키지와 개별 옵션에 같은 항목을 넣고 합계를 확인합니다. 지금은 실패해야 정상입니다.	/tests
02	고칠 범위가 두 루프에 걸쳐 있으므로 계획을 먼저 받습니다. 어느 쪽을 손댈지 읽고 나서 실행을 맡깁니다.	/plan
03	코드를 고칩니다. 생성된 코드가 지시문 파일의 규약을 따르는지 함께 봅니다.	Agent와 지시문 파일
04	01의 테스트가 통과하면 <code>npm test</code> 를 전체로 돌립니다. Hands-On Lab 2의 회귀 테스트가 실패하는지 봅니다.	<code>npm test</code>

※ 04에서 회귀 테스트가 실패하면 버그가 있던 금액을 정답으로 적어 둔 것이므로 기대값을 새 금액으로 고칩니다. 실패하지 않으면 그 조합을 안 덮은 것이니 그대로 두십시오.

막혔을 때의 조치

증상

조치

테스트가 실패하지 않음 패키지와 개별 옵션에 같은 항목을 넣었는지 확인합니다.

다른 테스트가 실패함 정상입니다. Hands-On Lab 2의 회귀 테스트가 이전 동작을 고정하고 있습니다.

규약이 안 지켜짐 지시문이 너무 추상적일 수 있습니다. 구체적인 문장으로 바꿔 보십시오.

변경이 너무 커짐 되돌리고 더 작게 나눠 다시 시킵니다.

체크리스트

확인	내용
테스트	고치기 전에 실패하는 테스트를 먼저 썼습니까.
지시문	생성된 코드가 지시문 파일에 적어 둔 규약을 따르고 있습니까.
계획	범위가 넓은 작업에 Plan을 써 보셨습니까.
Context	대화가 길어졌을 때 정리하거나 새 세션을 열어 보셨습니까.
전체 테스트	npm test 가 전부 통과합니까.

테스트와 계획

정답은 코드가 아니라 요청입니다. 01단계는 고치지 말라고 못 박는 것이 핵심입니다.

01단계 - 실패하는 테스트만 만들게 합니다

/tests #quote.js 의 calculateQuote 에 버그가 있습니다. 패키지에 포함된 옵션을 개별로도 고르면 같은 금액이 두 번 더해집니다. 이 문제를 드러내는 테스트를 test/duplicate-option.test.js 에 만들어 주세요. 기대값은 직접 계산하지 말고 패키지만 고른 결과와 비교하게 해 주세요. 코드는 아직 고치지 마세요.

02단계 - 정할 것을 사람에게 넘기게 합니다

/plan #quote.js 의 옵션 중복 계산을 고칠 계획을 세워 주세요. 겹친 옵션을 어느 가격으로 칠지는 직접 정하지 말고 질문으로 남겨 주세요. 기존 테스트에 미칠 영향도 함께 적어 주세요.

※ 01단계에서 기대값을 계산하지 말라는 문장을 빼면 Agent가 금액을 손으로 계산해 적습니다. 그러면 그 값이 맞는지 사람이 다시 확인해야 합니다.

수정과 회귀 테스트

03단계는 계획대로 고치게 하고, 04단계는 실패한 회귀 테스트를 어떻게 다룰지 시킵니다.

03단계 - 고칠 범위를 못 박습니다

계획대로 고쳐 주세요. 패키지에 포함된 옵션은 패키지 가격으로만 계산하고, 개별 옵션 목록에서는 건너뛰게 해 주세요.
테스트 파일은 고치지 마세요.

04단계 - 실패가 이번 수정 때문인지 먼저 가립니다

`npm test` 를 돌려 실패한 테스트를 알려 주세요. 실패한 것이 이번 수정 때문인지 확인하고, 맞다면 기대값을 새 금액으로 고쳐 주세요.
왜 바꾸는지 주석으로 남겨 주세요.

주문	수정 전	수정 후
테크 패키지 + 헤드업 개별	43,895,694원	42,747,624원
테크 패키지 + 헤드업 + 선루프	45,477,219원	44,329,149원

※ 04단계에서 실패 원인을 먼저 가리게 하는 것이 중요합니다. 이 문장이 없으면 Agent가 통과시키려고 숫자만 맞춰 버립니다.

같은 과제, 갈라진 길

01

결과 공유

두세 분이 화면을 띄우고 무엇을 만들었는지 말합니다.

02

어디서 갈렸는가

계획을 먼저 받은 분과 바로 코드를 시킨 분, 테스트를 먼저 쓴 분과 나중에 붙인 분. 같은 과제에서 갈라진 지점이 오늘의 요약입니다.

오늘 쓴 기능 한 장 정리

기능	설명
Code Completion	지금 쓰고 있는 줄의 다음 몇 줄을 제안합니다. 커서 위의 주석과 함수 이름이 요청 역할을 하므로, 무엇을 만들지 한 줄 적고 시작하면 정확도가 올라갑니다.
Next edit suggestions	한 곳을 고치면 같은 이유로 고쳐야 할 다른 자리를 짚어 줍니다. Tab 으로 그 자리까지 따라갑니다.
Ask, Plan, Agent	일의 성격에 따라 고르는 세 가지 Role입니다. 읽고 이해할 때는 Ask, 범위가 넓어 계획이 먼저 필요할 때는 Plan, 고칠 파일이 정해졌을 때는 Agent를 씁니다.
# 참조	요청에 넣을 대상을 직접 지정합니다. 파일을 알 때 붙이면 Agent가 찾는 과정이 빠지고, #codebase 는 파일을 모를 때만 씁니다.
/ 명령	자주 하는 요청을 짧게 부릅니다. 오늘 /init, /plan, /tests, /fix, /compact 를 배웠습니다.
지시문 파일	매 요청에 자동으로 들어가는 규약입니다. 프로젝트 전체는 AGENTS.md 나 copilot-instructions.md 에, 파일 종류별 규칙은 *.instructions.md 에 적습니다.
Agent Skills	가끔 하지만 절차가 고정된 작업을 폴더 하나로 묶어 둡니다. 요청이 맞을 때만 읽히므로 늘 Context를 차지하지는 않습니다.
MCP	IDE 밖의 시스템을 Tool로 연결합니다. .vscode/mcp.json 에 서버를 적으면 그 서버의 Tool이 목록에 들어옵니다.

WRAP-UP

요청 하나를 보낼 때 정하는 다섯 가지

오늘 배운 것을 실제 코드에 옮길 때 요청마다 마주치는 갈림길입니다.

항목

판단 기준

Role

고칠 파일과 방법이 정해졌으면 Agent. 어디부터 볼지 모르면 Plan. 남의 코드를 읽어야 하면 Ask

범위

diff를 눈으로 다 읽을 수 있는 크기로 끊습니다. 읽다 포기하면 이미 큰 것입니다.

규칙을 둘 곳

늘 지켜야 하면 AGENTS.md, 파일 종류마다 다르면 *.instructions.md 의 applyTo, 반복해 보내는 요청이면 *.prompt.md, 가끔 꺼내 쓰는 절차 묶음이면 Agent Skill

세션

주제가 바뀌면 새 세션을 엽니다. 같은 주제에서 길어졌을 때만 /compact

모델

기계적인 변환은 가벼운 모델로 충분합니다. 여러 단계에 걸친 판단이 필요할 때만 무거운 모델로 바꿉니다.

※ 다섯 줄을 매번 미리 정하라는 뜻은 아닙니다. 결과가 어긋났을 때 어느 줄을 잘못 골랐는지 되짚는 목록으로 쓰십시오.



심화 참고자료 — Copilot 활용 팁

Custom agents와 Subagent

지시문은 지켜 주기를 바라는 규약이고, Custom agent는 역할과 쓸 Tool과 모델을 `.agent.md` 파일 하나로 굳혀 둔 팀원입니다.

구분	내용
한 번만 정의	역할에 필요한 지시와 맥락을 파일 하나에 한 번만 적어 둡니다. 이후에는 그 Agent를 고르는 것만으로 같은 조건이 재현되므로, 매 요청마다 같은 지시문을 다시 쓰지 않습니다.
Tool 제한	지시문은 이렇게 해 달라고 권고하는 수단이지만, <code>tools</code> 에서 편집 Tool을 빼는 것은 아예 못 하게 만드는 강제 수단입니다. 조사와 검증만 맡길 Agent에서 이 차이가 결정적으로 작용합니다.
모델 고정	그 역할이 쓸 모델을 <code>model</code> 에 적어 둡니다. 가벼운 확인 작업을 가벼운 모델로 굳혀 두면 쓰는 사람이 매번 모델을 바꾸는 것을 기억하지 않아도 됩니다.
역할 잇기	<code>agents</code> 로 Sub Agent 호출을, <code>handoffs</code> 로 다음 단계 전환을 정의합니다. 계획에서 구현으로, 구현에서 리뷰로 넘기는 순서가 개인의 습관이 아니라 파일로 고정된 팀 표준이 됩니다.
팀 공유	<code>.github/agents/</code> 에 두면 저장소를 받는 사람 모두가 같은 Agent를 쓰고, <code>~/.copilot/agents/</code> 는 개인 전용입니다. 이름이 겹치면 홈 디렉터리 쪽이 우선합니다.

※ 파일이 실제로 어떻게 생겼는지는 다음 장에서 두 개를 나란히 봅니다.

Custom agent 파일 두 개의 구성

위쪽 --- 사이의 frontmatter가 이 Agent의 조건을 정합니다.
그 아래 본문에는 역할에게 그대로 전달할 지시문을 적습니다.

REVIEWER.AGENT.MD

편집 Tool을 주지 않은 검증 전용

```
---
name: Reviewer
description: 바뀐 코드의 정확성과 보안 검토
tools: ['search/codebase', 'read/readFile']
model: 'Claude Haiku 4.5'
---

판정마다 파일 경로와 줄 번호를 적고,
확신이 없으면 확인 필요로 분류하십시오.
```

BUILDER.AGENT.MD

Reviewer 를 Sub Agent로 부르는 쪽

```
---
name: Builder
description: 계획대로 구현한 뒤 검증에 인계
tools: ['edit', 'agent']
agents: ['Reviewer']
handoffs:
  - label: 검증으로 넘기기
    agent: Reviewer
    prompt: 방금 바꾼 파일만 검토하십시오
---
```

※ agents 는 Builder가 답변 도중 Reviewer를 직접 호출하는 통로이고, handoffs 는 답변이 끝난 뒤 사용자에게 뜨는 전환 버튼입니다. 서로를 대신하지 않으며, agents 를 쓰려면 tools 에 agent 를 함께 넣어야 합니다.

Subagent에 넘겨서 얻는 것

Sub Agent는 Main Agent의 Context를 물려받지 않은 채로 시작합니다. 이미 옳다고 전제해 둔 판단이 없으므로 같은 자리를 다시 의심합니다.

탐색 과정에서 읽은 파일과 검색 결과도 Sub Agent 쪽에 축적되고, Main Agent로 반환되는 것은 결론뿐입니다.

상황

이점

검증과 리뷰

Sub Agent는 Main Agent의 Context를 받지 않습니다. "내가 맞게 고쳤을 것"이라는 전제를 물려받지 않으므로, 편향을 차단하는 것이 가장 큰 이득입니다.

관점 나누기

정확성, 보안, 재현 가능성을 각각 맡기면 한 문맥에서 차례로 볼 때보다 덜 놓칩니다.

많이 읽는 작업

저장소 전반을 탐색하는 일을 위임하면 읽은 파일과 검색 결과가 Main Agent의 Context를 잠식하지 않습니다.

긴 작업

Main Agent의 Context Window가 덜 차기 때문에 요약 시점이 늦춰집니다. 초반에 지정한 조건이 그만큼 오래 유지됩니다.

※ 총 사용 토큰이 줄어드는 것은 아닙니다. Sub Agent도 자기 몫을 소비하므로 절약되는 것은 Main Agent의 Context Window입니다. Sub Agent는 맥락을 모르니 위임하는 문장에 필요한 조건을 다 적어야 합니다.

Agent Hook이 끼어드는 여덟 지점

Hook은 Agent가 도는 도중 정해진 시점에 내가 적어 둔 명령을 실행하는 장치입니다. 지시문은 지켜 주기를 바라는 쪽이고, Hook은 코드로 결과를 확정합니다.

VS Code에서는 명령 팔레트의 Chat: Configure Hooks 로 등록하며, 저장되는 파일은 다음 장에서 봅니다.

이벤트	시점	활용 예
SessionStart	새 세션의 첫 요청을 보낼 때	branch 이름 같은 프로젝트 정보를 요청에 자동으로 붙입니다
UserPromptSubmit	요청을 보낼 때마다	누가 무엇을 요청했는지 기록으로 남깁니다
PreToolUse	Agent가 Tool을 부르기 직전	위험한 명령을 막거나 승인을 요구합니다
PostToolUse	Tool이 성공으로 끝난 뒤	편집이 끝나면 formatter나 검사를 돌립니다
PreCompact	대화가 압축되기 직전	지워지면 곤란한 내용을 파일로 뽑아 둡니다
SubagentStart	Sub Agent가 시작될 때	Sub Agent에 얼마나 위임했는지 집계합니다
SubagentStop	Sub Agent가 종료될 때	결과를 모으고 남은 자원을 정리합니다
Stop	Agent 실행이 멈출 때	보고서를 만들거나 알림을 보냅니다

※ Preview 단계입니다. 공식 문서에 "The configuration format and behavior might change in future releases" 라고 적혀 있으므로, 사내 표준으로 굳히기 전에 각자 쓰는 VS Code 버전에서 다시 확인해야 합니다.

Hook 설정 파일의 위치와 첫 예제

Hook은 JSON 파일로 등록합니다. 저장소에 넣어 commit하면 Workspace 범위로 걸립니다.

범위	파일	적용 대상
Workspace	.github/hooks/*.json	저장소를 연 사람 모두. 팀 공통 규칙을 등록하는 위치입니다.
Workspace	.claude/settings.json	Claude harness의 설정 형식을 읽습니다. 다만 Tool 이름과 필드 표기가 달라 스크립트를 손봐야 합니다.
개인	~/.copilot/hooks, ~/.claude/settings.json	내 계정 전체. 저장소를 가리지 않고 걸립니다.
Custom agent	.agent.md frontmatter의 hooks	그 Agent로 도는 세션에만 걸립니다. <code>chat.useCustomAgentHooks</code> 를 켜야 동작하고, 이 기능은 따로 preview 단계입니다.

.github/hooks/guard.json - Tool을 부르기 전에 검사 스크립트를 태웁니다

```
{ "hooks": { "PreToolUse": [ { "type": "command", "command": "./scripts/guard.sh" } ] } }
```

scripts/guard.sh - 파괴적인 명령이면 거부 판정을 돌려줍니다

```
CMD=$(cat | jq -r 'select(.tool_name=="runTerminalCommand").tool_input.command // empty')
echo "$CMD" | grep -qE 'rm -rf|DROP TABLE' \
  && echo '{"hookSpecificOutput":{"permissionDecision":"deny"}}' || echo '{"continue":true}'
```

※ 여러 범위에 겹치면 Workspace 설정이 먼저 적용되고, PreToolUse 판정이 여럿이면 deny, ask, allow 순으로 가장 엄한 쪽이 적용됩니다.

지시문보다 강한 것

같은 규칙이라도 무엇으로 강제하느냐에 따라 지켜지는 정도가 다릅니다. 아래로 내려갈수록 빠져나갈 구멍이 넓어집니다. "any 를 쓰지 않는다"는 규칙은 지시문에 문장으로 적을 수도 있고, 컴파일러 설정으로 빌드가 실패하게 만들 수도 있습니다.

순위	강제 수단	빠져나갈 구멍
01	컴파일러와 타입 검사	거의 없습니다. 다만 타입 검사는 <code>@ts-ignore</code> 나 <code># type: ignore</code> 같은 억제 주석으로 한 줄씩 빠져나갑니다.
02	CI 파이프라인	필수 상태 검사로 걸어 두면 개인이 끌 수 없습니다. 저장소 관리자 권한은 기본값에서 우회되고, 알려 주는 시점도 늦습니다.
03	pre-commit hook	<code>git commit --no-verify</code> 로 넘어갑니다. 게이트가 아니라 빠른 알림으로 보는 편이 맞습니다.
04	Agent Hook의 PreToolUse	Agent가 부르는 Tool만 막습니다. 사람이 터미널에 직접 친 명령은 지나갑니다.
05	지시문 파일	지켜 주기를 바라는 것이지 막는 것이 아닙니다. 대화가 길어지면 흐려집니다.

※ 아래쪽이 쓸모없다는 뜻이 아닙니다. 위로 갈수록 확실하고, 아래로 갈수록 빨리 알려 줍니다. 같은 규칙을 위아래에 함께 걸어 두면 빨리 알려 주면서 결국 막는 규칙이 됩니다.

갈래가 여럿인 코드베이스

같은 일을 하는 코드가 저장소 안에 여러 형태로 남아 있으면 Agent가 어느 쪽을 따라야 할지 정할 수 없습니다. 같은 요청에서 결과가 갈라지는 이유가 여기 있습니다.

Java와 Spring 예 – 컨트롤러의 오류 처리가 세 갈래로 남아 있다면

- A 커스텀 예외를 던지고 `@ControllerAdvice` 가 받는다
- B `catch (Exception e)` 로 잡아 `ResponseEntity.status(500)` 을 직접 만든다
- C 예외를 삼키고 `null` 을 돌려준다

단계	하는 일
01	갈래 하나를 표준으로 정하고 나머지를 지웁니다. 남겨 두면 Agent가 계속 본보기로 삼습니다.
02	고치는 순서는 Hands-On Lab 2와 같습니다. 회귀 테스트를 먼저 만들고 손댁니다.
03	통일한 뒤에는 검사로 굳힙니다. Java라면 ArchUnit, Checkstyle, PMD가 어긋난 형태를 걸러 냅니다.

※ 규칙이 없어서 생기는 문제가 아니라 규칙이 여러 개 남아 있어서 생기는 문제입니다. 지시문에 "A로 하라"고 적어도 B와 C가 코드에 남아 있는 한 같은 일이 되풀이됩니다.

Agent 개발 환경에서 컴파일과 린트를 엄격하게 설정하는 것이 유리한 이유

옵션을 엄격하게 설정하면 고칠 곳이 한꺼번에 드러납니다.

예전에는 그 정리 비용 때문에 옵션을 느슨하게 두었지만, 이제는 그 정리를 Agent에 대량으로 맡길 수 있습니다.

01

예전에 느슨하게 둔 이유

비용 옵션을 엄격하게 바꾸면 사람이 수백 곳을 직접 고쳐야 했습니다. 그래서 경고를 남겨 둔 채로 넘어가는 편이 합리적이었습니다.

02

지금은 조일 수 있는 이유

비용 경고를 오류로 올린 뒤 남은 것을 Agent에 넘깁니다. 판단이 거의 필요 없는 기계적인 수정이라 80페이지 기준으로 가벼운 모델로도 됩니다.

03

따라오는 이득

정형화 코드가 한 가지 형태로 모이면 Agent가 따라갈 본보기가 하나로 좁혀집니다.

팀이 반복하는 절차를 Skill로 만드는 기준

35페이지에서 만든 것은 Skill 한 개짜리 예제였습니다. 실무에서는 팀이 반복하는 절차마다 Skill을 하나씩 만들어 두며, 그 수가 쌓일수록 효과가 커집니다.

항목	적는 내용
호출 조건	description에 그 작업을 요청할 때 쓸 만한 표현을 여러 가지로 적습니다. 한 가지만 적어 두면 다르게 표현한 요청에서는 이 Skill이 선택되지 않습니다.
절차	주간 보고서 작성이나 공지 등록처럼 순서가 정해진 여러 단계를 SKILL.md 본문에 고정합니다.
주의 사항	그 작업에서 매번 실수하는 지점을 함께 적습니다. 시간대 변환이나 권한 확인처럼 사람이 반복해서 빠뜨리는 항목이 대상입니다.
함께 두는 파일	양식과 지난 산출물을 같은 폴더에 둡니다. 폴더 안의 파일도 Copilot이 찾아내지만, SKILL.md 본문에서 링크로 연결해 두어야 필요한 시점에 확실히 참조합니다.

※ Java와 Spring이라면 "엔티티에 컬럼 추가" 하나를 Skill로 만들어 두는 것이 좋은 출발점입니다. 마이그레이션과 Entity, DTO, 매핑, 테스트를 늘 같은 순서로 처리하게 됩니다.

테스트를 시킬 때 걸리는 두 함정

"테스트를 잘 만들어 주세요" 로는 부족합니다. 두 가지가 반복해서 걸립니다.

함정	벌어지는 일	막는 방법
지금 값이 정답	버그가 있는 현재 동작을 그대로 기대값에 박아 넣습니다. 테스트는 통과하지만 버그가 고정됩니다.	76페이지 요청처럼 기대값을 직접 계산하지 말라고 적고, 무엇과 비교할지 함께 지정합니다.
정상 경로만	잘 되는 경우 하나만 만들고 끝냅니다. 경계값과 예외는 건드리지 않습니다.	테스트를 쓰기 전에 정상, 경계값, 예외, 잘못된 입력 네 가지를 나열하게 하고 목록부터 확인합니다.

※ 이름을 `rateLimitNeverGoesNegative` 처럼 시나리오가 읽히도록 짓습니다. 실패했을 때 무엇이 깨졌는지 로그만 보고도 알 수 있습니다.

감사합니다.

S.Pin Technology